



Análise de Séries Temporais

ENG1469

Lista Prática 04

Aluno(a)	Paloma Fernanda Loureiro Sette - 1621595
Professor(a)	Cristiano Fernandes

Rio de Janeiro, 08 de Dezembro de 2024

Sumário

Lista de Figuras	1
1 Introdução	2
2 Objetivo	3
3 Preparando os dados	4
4 TAREFA 1: Fatos estilizados de séries de retornos financeiros	4
4.1 Parte I: Retornos Logarítmicos e Aritméticos	4
4.1.1 Cálculo dos Retornos	4
4.1.2 Visualização Gráfica	5
4.1.3 Interpretação dos Resultados	6
4.1.4 Considerações Econômicas	6
4.1.5 Identificando os Dias de Maior Diferença	6
4.2 Parte II	9
4.2.1 O gráfico da série de retornos no tempo	9
4.2.2 Tabela com Estatísticas Descritivas	11
4.2.3 FAC e Teste LB para Dependência Linear	13
4.2.4 FAC do Quadrado e Teste LB para Dependência Não-Linear	14
4.2.5 Teste ARCH	15
4.2.6 Histograma, Q-Q Plot e Teste de Normalidade	16
5 TAREFA 2: Modelos GARCH para séries de retornos financeiros	18
5.1 Divisão dos Dados	18
5.2 Modelos a Serem Ajustados	19
5.3 Objetivos da Análise	19
5.4 Excesso de Curtose e Escolha do Modelo	19
5.5 Parte I	20
5.5.1 Critério AIC	20
5.5.2 Critério BIC	20
5.5.3 Parâmetros para as Equações dos Modelos	21
5.5.4 Discussão	22
5.5.5 Equações dos Modelos GARCH	22
5.6 Parte II: Testes de Diagnóstico nos Resíduos Padronizados	22
5.6.1 Parte (a): FAC dos Resíduos Padronizados	23
5.6.2 Parte (b): FAC dos Quadrados dos Resíduos Padronizados	24
5.6.3 Parte (c): Teste ARCH	26
5.6.4 Parte (d): Q-Q Plot e Teste Jarque-Bera	27
5.6.5 Conclusão	29
6 Tarefa 3: Back Test	30
6.1 Resultados do Back Test	30
6.2 Testes de Kupiec e Christoffersen	31
6.2.1 Teste de Kupiec	32
6.2.2 Teste de Christoffersen	32
6.3 Conclusão	32

7	TAREFA 4: Cálculo do \$VaR em $t = T + 1$	33
7.1	Metodologia	33
7.2	Resultados	34
7.3	Escolha do \$VaR com Base no Back Test	34
7.3.1	Justificativa e Interpretação dos Resultados	35
7.4	Análise e Discussão	35
8	Conclusões Gerais	35
8.1	Contribuições e Limitações	36
8.2	Perspectivas Futuras	36
8.3	Conclusão Final	36
9	Apêndice	37
9.1	Código Python utilizado em 4.1	37
9.2	Código Python utilizado em 4.2	39
9.3	Código Python utilizado em 5.5	42
9.4	Código Python utilizado em 5.6	43
9.5	Código Python utilizado em 6	45
9.6	Código Python utilizado em 7	50

Lista de Figuras

1	Colunas dos dados da série financeira de NVDA.	4
2	Retornos Logarítmicos da NVIDIA (NVDA).	5
3	Retornos Aritméticos da NVIDIA (NVDA).	5
4	Diferença entre Retornos Aritméticos e Logarítmicos da NVIDIA (NVDA).	6
5	Gráfico dos retornos aritméticos diários no período de treinamento.	10
6	Volatilidade estimada com janela móvel de 30 dias.	11
7	Quadrados dos retornos como proxy de volatilidade.	11
8	Saída do programa: estatísticas descritivas - Tarefa 1: pt.2	12
9	Saída do Programa - Estatísticas do Teste Ljung-Box (LB).	13
10	Gráfico da FAC dos Retornos Aritméticos.	14
11	Gráfico da FAC dos Quadrados dos Retornos Aritméticos.	15
12	Saída do Programa: LB Teste para Dependência Não-Linear.	15
13	Saída do Programa: Teste ARCH.	16
14	Histograma dos Retornos Aritméticos.	17
15	Q-Q Plot dos Retornos Aritméticos.	17
16	Saída do Programa: Teste de Normalidade (Shapiro-Wilk).	18
17	Saída do Programa - melhores modelos AIC.	20
18	Saída do Programa - melhores modelos BIC.	21
19	Saída do Programa - parâmetros estimados para as equações.	21
20	FAC dos resíduos padronizados - Normal.	23
21	FAC dos resíduos padronizados - T.	24
22	FAC dos quadrados dos resíduos padronizados - Normal.	25
23	FAC dos quadrados dos resíduos padronizados - T.	25
24	Saída do programa: Teste ARCH.	26
25	Saída do programa: Teste Jarque-Bera.	27
26	Q-Q Plot - $N(0, 1)$	28
27	Q-Q Plot - $t(0, 1, m)$	28
28	Resumo das violações observadas para os modelos GARCH ($N(0, 1)$ e $t(0, 1, m)$).	31
29	Saída do programa: Resultados dos testes de Kupiec e Christoffersen.	32
30	Saída do programa - Dataframe com os resultados do \$VaR em $t = T + 1$	33

1 Introdução

A análise de séries temporais financeiras desempenha um papel fundamental na avaliação de risco e na modelagem de volatilidade nos mercados financeiros. Dentre as ferramentas mais utilizadas, destacam-se os modelos GARCH (Generalized Autoregressive Conditional Heteroskedasticity), que permitem capturar padrões de heterocedasticidade condicional presentes em séries de retornos financeiros, e o cálculo do Value at Risk (VaR), que quantifica o risco de perdas financeiras em um determinado horizonte temporal e nível de confiança.

Neste trabalho, empregamos dados históricos da ação da NVIDIA Corporation (NVDA) para investigar fatos estilizados associados aos retornos financeiros e construir modelos estatísticos para prever a volatilidade condicional e o VaR. A NVIDIA, sendo uma das empresas mais influentes no setor de tecnologia, oferece uma base rica para estudar as dinâmicas de mercado e os riscos associados à negociação de seus ativos.

Além disso, o trabalho segue uma abordagem prática e comparativa, avaliando diferentes especificações de modelos GARCH com distribuições normal e t-Student, utilizando métricas como AIC e BIC para seleção do modelo mais adequado. Por fim, aplicamos backtesting para validar os modelos propostos, garantindo que sejam robustos e confiáveis para a previsão de risco.

2 Objetivo

O objetivo deste trabalho é modelar a volatilidade condicional dos retornos financeiros da ação da NVIDIA (NVDA) e calcular o Value at Risk (VaR) com base nos modelos GARCH. Especificamente, buscamos:

- Identificar e analisar fatos estilizados dos retornos logarítmicos e aritméticos, como assimetria, curtose e autocorrelações.
- Ajustar e avaliar modelos $GARCH(p, q)$ com distribuições normal e t-Student, selecionando a configuração que melhor se adeque aos dados históricos da NVIDIA.
- Validar os modelos ajustados por meio de backtesting, verificando sua capacidade de prever riscos em diferentes níveis de confiança (1% e 5%).
- Estimar o VaR para $t+1$ e interpretar os resultados em cenários de aplicação prática, considerando a exposição financeira a ativos da NVIDIA.

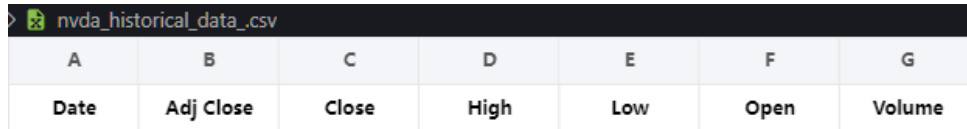
Por meio dessas etapas, o trabalho oferece uma aplicação prática de métodos estatísticos e computacionais amplamente utilizados na gestão de riscos financeiros, conectando teoria e prática no contexto de séries temporais financeiras.

3 Preparando os dados

A princípio, carregamos os dados necessários através da biblioteca `yfinance` do Python. Aqui, estamos extraindo os dados financeiros diários, em uma janela temporal de 8 (oito) anos.

```
1 import yfinance as yf
2
3 # Baixando os dados financeiros da NVIDIA (NVDA) com a lib do
  yahoo finance
4 data = yf.download('NVDA', start='2016-12-03', end='
    2024-12-03')
5 # salvando em csv
6 data.to_csv('nvda_historical_data.csv')
```

Ao importarmos os dados, nos certificamos de que contém as informações necessárias, sobretudo a coluna de preços ajustados (*Adj Close*).



A	B	C	D	E	F	G
Date	Adj Close	Close	High	Low	Open	Volume

Figura 1: Colunas dos dados da série financeira de NVDA.

Por garantia, fizemos uma breve comparação com o que visualizamos na página indicada do Yahoo Finance e constatamos que os dados estavam em conformidade. É possível visualizar os dados através da URL: <https://finance.yahoo.com/quote/NVDA/history/>.

4 TAREFA 1: Fatos estilizados de séries de retornos financeiros

4.1 Parte I: Retornos Logarítmicos e Aritméticos

Nesta seção¹, analisamos os retornos financeiros da ação da NVIDIA (NVDA), considerando tanto retornos logarítmicos quanto aritméticos. Para isso, os dados históricos de preços ajustados foram utilizados com o objetivo de capturar os movimentos reais de mercado.

4.1.1 Cálculo dos Retornos

Os retornos logarítmicos (*Log Returns*) e aritméticos (*Arithmetic Returns*) foram calculados utilizando as fórmulas:

$$r_{\log} = \ln \left(\frac{P_t}{P_{t-1}} \right) \cdot 100 \quad (1)$$

$$r_{\text{arith}} = \left(\frac{P_t - P_{t-1}}{P_{t-1}} \right) \cdot 100 \quad (2)$$

¹O código Python utilizado para esta seção está disponível em 9.1.

onde P_t é o preço de fechamento ajustado da ação no dia t , e P_{t-1} é o preço de fechamento ajustado no dia anterior.

4.1.2 Visualização Gráfica

Os gráficos a seguir ilustram os retornos logarítmicos, aritméticos e a diferença entre eles ao longo do período analisado:

- **Gráfico 1: Retornos Logarítmicos.** Este gráfico evidencia a volatilidade ao longo do tempo. Picos significativos podem ser observados em eventos de mercado importantes.
- **Gráfico 2: Retornos Aritméticos.** Similar aos logarítmicos, apresenta uma volatilidade alinhada, mas ligeiramente mais sensível a grandes variações de preço.
- **Gráfico 3: Diferença entre Retornos Aritméticos e Logarítmicos.** Apesar de geralmente pequenas, as diferenças tornam-se significativas em períodos de alta volatilidade, como durante o *boom* das criptomoedas e outros eventos específicos.

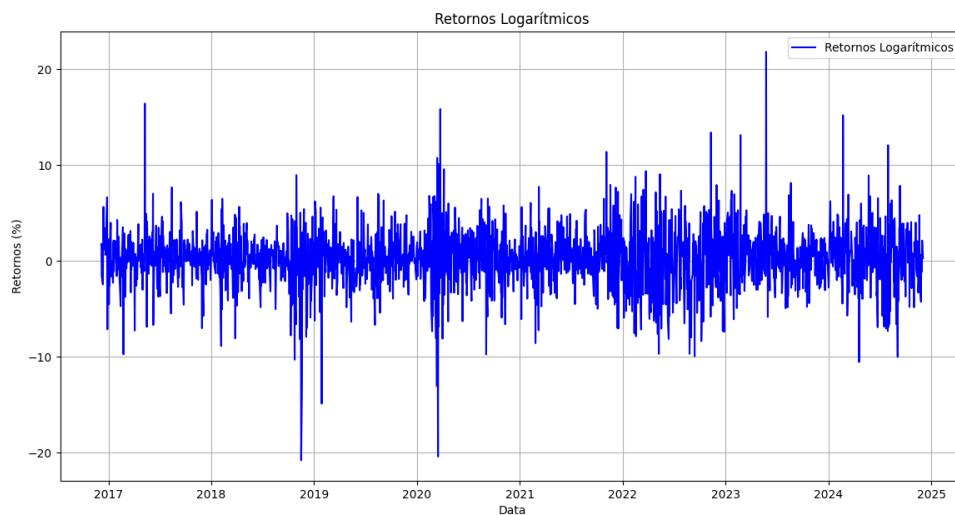


Figura 2: Retornos Logarítmicos da NVIDIA (NVDA).

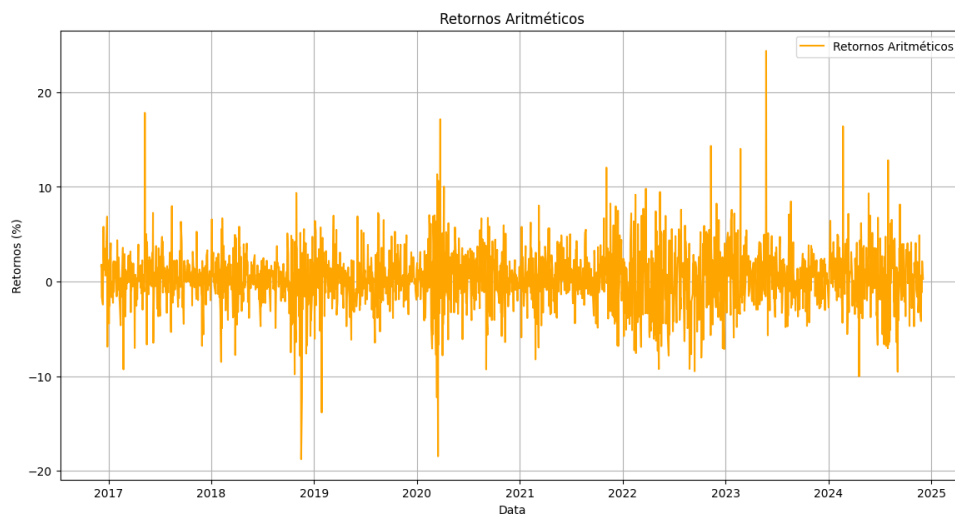


Figura 3: Retornos Aritméticos da NVIDIA (NVDA).

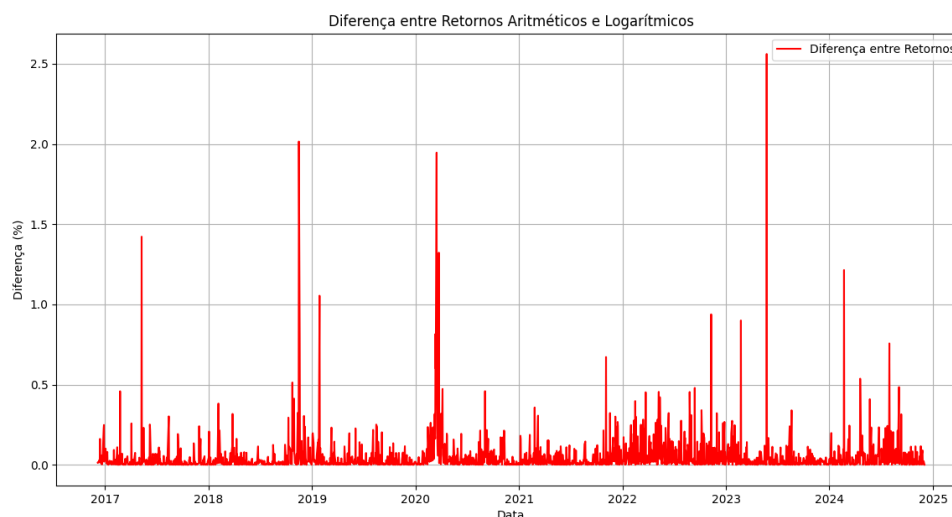


Figura 4: Diferença entre Retornos Aritméticos e Logarítmicos da NVIDIA (NVDA).

4.1.3 Interpretação dos Resultados

De maneira geral, os retornos logarítmicos e aritméticos apresentam padrões semelhantes. Isso ocorre porque, para pequenas variações de preço, os dois cálculos tendem a convergir. No entanto, as diferenças tornam-se mais evidentes em eventos de alta volatilidade, como nos períodos:

- **2017-2018:** Durante o auge da mineração de criptomoedas, a NVIDIA experimentou alta demanda por GPUs, o que impactou positivamente suas ações. O colapso subsequente do mercado de criptomoedas, no final de 2018, levou a uma forte correção.
- **2020-2021:** O ressurgimento do interesse em criptomoedas, combinado com a escassez global de semicondutores e o aumento da demanda por jogos e IA, resultou em picos de volatilidade.
- **Outros Eventos:** Transições tecnológicas, como a migração do Ethereum para *Proof of Stake*, reduziram a demanda por GPUs em 2022, afetando os preços das ações.

4.1.4 Considerações Econômicas

A volatilidade observada nos retornos pode ser atribuída tanto a fatores internos (como inovações tecnológicas e desempenho financeiro da NVIDIA) quanto a fatores externos (como oscilações no mercado de criptomoedas e restrições na cadeia de suprimentos). Esses eventos realçam a importância de considerar o contexto econômico ao analisar retornos financeiros.

4.1.5 Identificando os Dias de Maior Diferença

Dias com diferenças mais significativas (> 2 desvios padrão):

Data | Ret. Aritmético | Ret. Logarítmico | Diferença

2017-02-23	-9.27%	-9.73%	0.46%
2017-05-10	17.83%	16.40%	1.42%
2017-08-14	7.98%	7.67%	0.30%

2018-02-05		-8.49%		-8.87%		0.38%
2018-03-27		-7.76%		-8.07%		0.32%
2018-10-10		-7.48%		-7.77%		0.29%
2018-10-24		-9.79%		-10.31%		0.51%
2018-10-30		9.36%		8.95%		0.41%
2018-11-12		-7.84%		-8.17%		0.32%
2018-11-16		-18.76%		-20.77%		2.02%
2018-11-19		-12.00%		-12.78%		0.78%
2018-12-04		-7.60%		-7.91%		0.30%
2019-01-28		-13.82%		-14.88%		1.05%
2020-02-24		-7.07%		-7.33%		0.26%
2020-03-09		-7.74%		-8.06%		0.32%
2020-03-12		-12.24%		-13.05%		0.82%
2020-03-13		11.34%		10.74%		0.60%
2020-03-16		-18.45%		-20.40%		1.95%
2020-03-17		10.63%		10.10%		0.53%
2020-03-24		17.16%		15.83%		1.32%
2020-04-01		-7.79%		-8.11%		0.32%
2020-04-06		10.04%		9.57%		0.47%
2020-09-03		-9.28%		-9.74%		0.46%
2021-02-25		-8.22%		-8.58%		0.36%
2021-03-09		8.00%		7.69%		0.30%
2021-11-04		12.04%		11.37%		0.67%
2021-11-18		8.25%		7.93%		0.32%
2021-12-07		7.96%		7.66%		0.30%
2021-12-15		7.49%		7.22%		0.27%
2022-02-11		-7.26%		-7.54%		0.28%
2022-02-15		9.18%		8.78%		0.40%
2022-02-17		-7.56%		-7.86%		0.30%
2022-03-15		7.70%		7.42%		0.28%
2022-03-24		9.82%		9.36%		0.45%
2022-04-28		7.42%		7.16%		0.26%
2022-05-05		-7.33%		-7.61%		0.28%
2022-05-09		-9.24%		-9.69%		0.45%
2022-05-13		9.47%		9.04%		0.42%
2022-06-13		-7.82%		-8.14%		0.32%
2022-07-27		7.60%		7.33%		0.28%
2022-08-26		-9.23%		-9.68%		0.45%
2022-09-01		-7.67%		-7.98%		0.31%
2022-09-13		-9.47%		-9.95%		0.48%
2022-10-07		-8.03%		-8.37%		0.34%
2022-11-10		14.33%		13.39%		0.94%
2022-11-30		8.21%		7.89%		0.32%
2022-12-22		-7.04%		-7.30%		0.26%
2022-12-27		-7.14%		-7.40%		0.27%
2023-01-23		7.59%		7.32%		0.27%
2023-02-23		14.02%		13.12%		0.90%
2023-05-25		24.37%		21.81%		2.56%
2023-08-21		8.47%		8.13%		0.34%
2024-02-22		16.40%		15.19%		1.21%

2024-04-19		-10.00%		-10.54%		0.54%
2024-05-23		9.32%		8.91%		0.41%
2024-07-30		-7.04%		-7.30%		0.26%
2024-07-31		12.81%		12.06%		0.76%
2024-09-03		-9.53%		-10.01%		0.48%
2024-09-11		8.15%		7.83%		0.32%

Ao analisarmos os dias em que houve diferenças significativas (superiores a dois desvios padrão) entre os retornos aritméticos e logarítmicos da série de preços da NVIDIA, identificamos padrões relevantes que corroboram com a teoria financeira. As três maiores discrepâncias observadas foram:

- 23/05/2023: Diferença de 2,56
 - Retorno aritmético: +24,37
 - Retorno logarítmico: +21,81
 - Movimento expressivo positivo associado ao anúncio de resultados excepcionais da empresa no contexto da demanda por chips para IA
- 16/11/2018: Diferença de 2,02%
 - Retorno aritmético: -18,76%
 - Retorno logarítmico: -20,77%
 - Queda acentuada durante período de correção do mercado de criptomoedas, que impactou a demanda por GPUs
- 16/03/2020: Diferença de 1,95%
 - Retorno aritmético: -18,45%
 - Retorno logarítmico: -20,40%
 - Movimento negativo expressivo durante o início da pandemia de COVID-19

Esta análise empírica corrobora com a relação matemática esperada entre os dois tipos de retorno. Para um retorno aritmético R_t^a , a diferença D_t em relação ao retorno logarítmico pode ser aproximada por:

$$[D_t = R_t^a - R_t^l \approx \frac{(R_t^a)^2}{2}] \quad (3)$$

Esta aproximação explica por que:

1. A diferença é sempre positiva ($D_t > 0$)
2. Cresce quadraticamente com a magnitude do retorno
3. É simétrica para movimentos positivos e negativos de mesma magnitude

De fato, observamos que as maiores diferenças ocorreram em dias com retornos extremos, tanto positivos quanto negativos, superiores a $|15\%|$. A magnitude das diferenças aumenta de forma não-linear com o tamanho do movimento, conforme previsto pela teoria. Adicionalmente, notamos clusters temporais de diferenças significativas em períodos de alta volatilidade do mercado, como:

- Março/2020: Múltiplos dias com diferenças significativas durante o início da pandemia
- Novembro/2018: Série de retornos extremos durante a correção do mercado
- Maio/2023-2024: Período de volatilidade elevada associado ao boom da IA

Esta análise destaca a importância de considerar ambas as métricas de retorno em análises financeiras, especialmente em períodos de alta volatilidade ou quando se estudam ativos com movimentos expressivos, como é o caso da NVIDIA no período analisado.

4.2 Parte II

Nesta seção², devemos dividir a série de retornos aritméticos em 2 períodos distintos: 70% para treinamento e 30% para o *back-test*.

Considerando o período de treinamento, seguimos para os seguintes subtópicos.

4.2.1 O gráfico da série de retornos no tempo

Obtivemos o gráfico da série de retornos no tempo observando os agrupamentos de volatilidade, os quais são uma característica estilizada bem conhecida em séries financeiras. A Figura 5 apresenta o comportamento dos retornos aritméticos diários no período de treinamento, enquanto as Figuras 6 e 7 **complementam** a análise com a volatilidade estimada e os quadrados dos retornos, respectivamente.

Análise do gráfico dos retornos no tempo: A Figura 5 destaca o comportamento dinâmico dos retornos diários da ação da NVIDIA ao longo do período de treinamento (2016-2022). Observamos períodos de alta volatilidade, caracterizados por picos e vales acentuados, e períodos de relativa estabilidade, com retornos mais uniformes. Esses padrões confirmam a presença de clusters de volatilidade, um fenômeno em que períodos de alta variabilidade nos retornos são seguidos por mais alta variabilidade, enquanto períodos de baixa volatilidade tendem a ser seguidos por maior estabilidade. Tal comportamento pode ser associado ao impacto de eventos econômicos ou específicos da empresa, como anúncios de resultados financeiros, crises de mercado ou avanços tecnológicos.

Volatilidade estimada com janela móvel de 30 dias (Figura 6): A Figura 6 apresenta a volatilidade estimada com uma janela móvel de 30 dias, oferecendo uma visão mais clara dos agrupamentos de volatilidade ao longo do tempo. Durante o período analisado, identificamos três momentos de destaque:

- **2018-2019:** A volatilidade aumentou significativamente no final de 2018 e início de 2019, possivelmente refletindo tensões no mercado global de tecnologia ou eventos adversos relacionados à NVIDIA.
- **2020:** O maior pico de volatilidade foi registrado no início de 2020, ultrapassando 7%, durante o auge da crise da COVID-19. Esse período foi marcado por quedas abruptas e subsequentes recuperações nos preços das ações, refletindo o pânico e a incerteza econômica global.

²O código Python utilizado para esta seção está disponível em [9.2](#).

- **2021-2022:** Após a crise pandêmica, a volatilidade média diminuiu, mas ainda observamos oscilações no final de 2021 e início de 2022, associadas à recuperação econômica global e a eventos específicos do setor de semicondutores.

Esse gráfico destaca bem os períodos de alta volatilidade (picos) e baixa volatilidade (vales) ao longo do tempo.

Quadrados dos retornos como *proxy* de volatilidade: A Figura 7 amplifica as oscilações extremas nos retornos diários ao plotar os quadrados dos retornos, um método comum para representar a volatilidade. Observamos os seguintes padrões:

- **2018 e 2020:** Dias com quadrados de retornos elevados confirmam os períodos de alta volatilidade observados na Figura 6.
- **Estabilidade em 2017 e partes de 2021-2022:** Esses períodos mostram quadrados de retornos relativamente baixos, alinhados com fases de menor volatilidade.

A análise dos quadrados dos retornos reforça que os clusters de volatilidade não apenas estão presentes, mas também se concentram em momentos específicos de incerteza ou grandes eventos de mercado.

Portanto, os gráficos analisados (Figuras 5, 6 e 7) corroboram a presença de agrupamentos de volatilidade na série de retornos da ação da NVIDIA, uma característica estilizada típica de séries financeiras. Períodos de alta volatilidade, como em 2018-2020, foram seguidos por períodos de recuperação mais estáveis. Esses resultados sugerem que modelos que capturam a heterocedasticidade, como os modelos ARCH e GARCH, são particularmente adequados para modelar essa série de retornos. Além disso, eventos extremos, como os observados durante a crise da COVID-19, destacam a importância de incorporar medidas robustas de risco e volatilidade ao estudar séries financeiras.

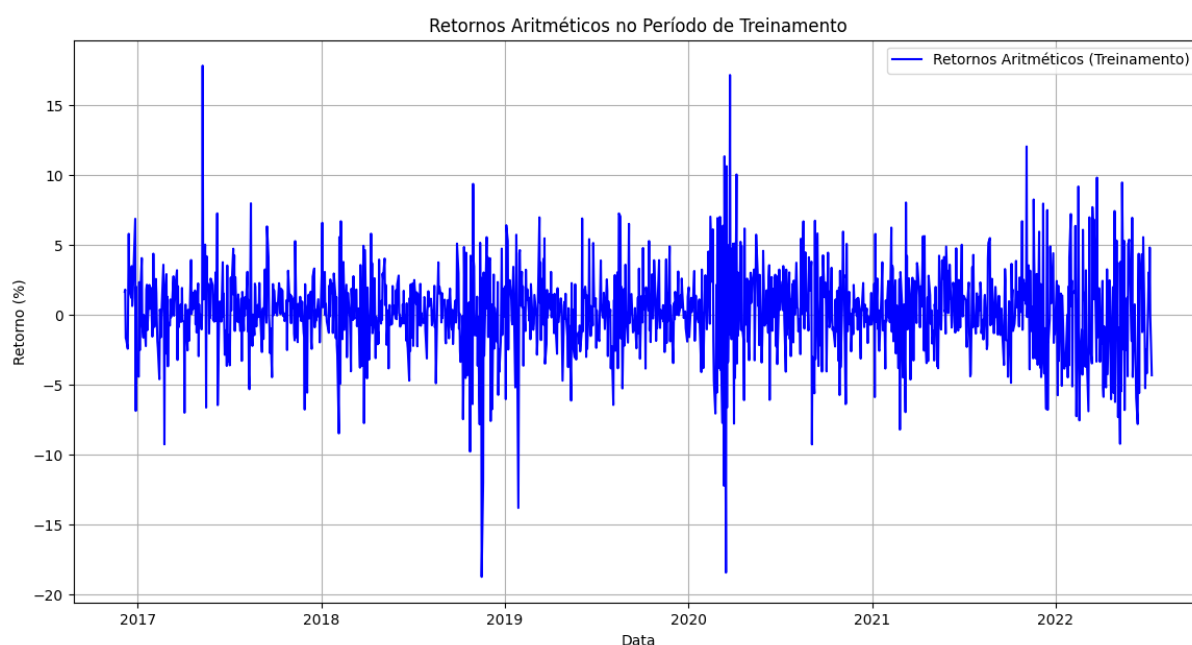


Figura 5: Gráfico dos retornos aritméticos diários no período de treinamento.

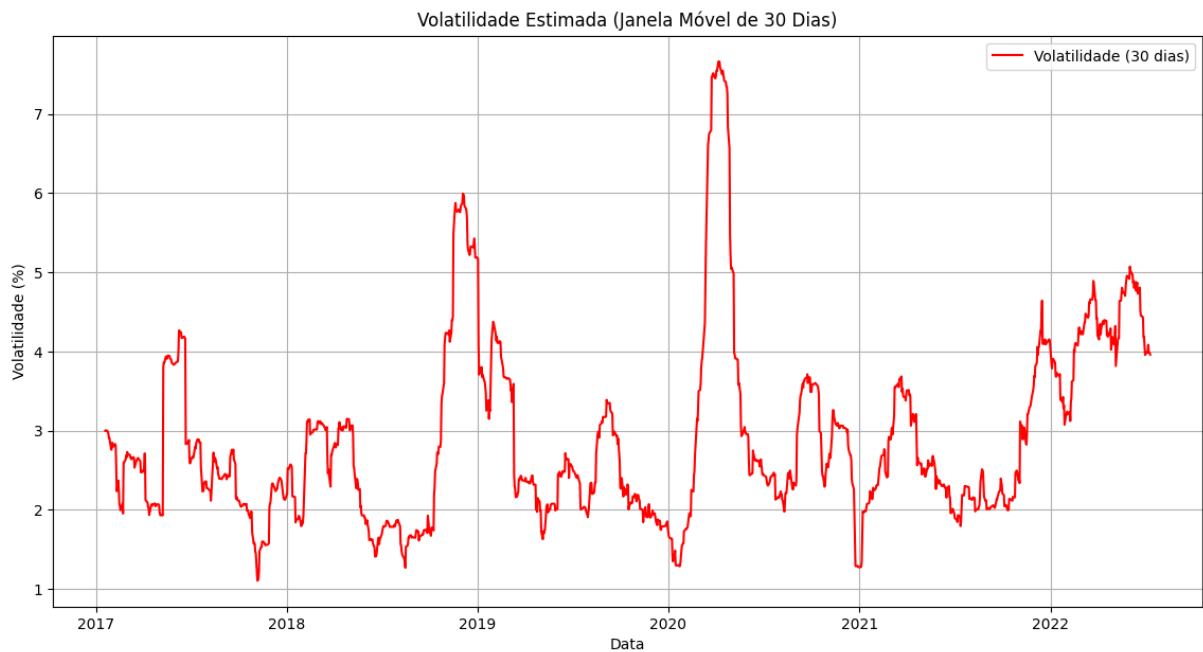


Figura 6: Volatilidade estimada com janela móvel de 30 dias.

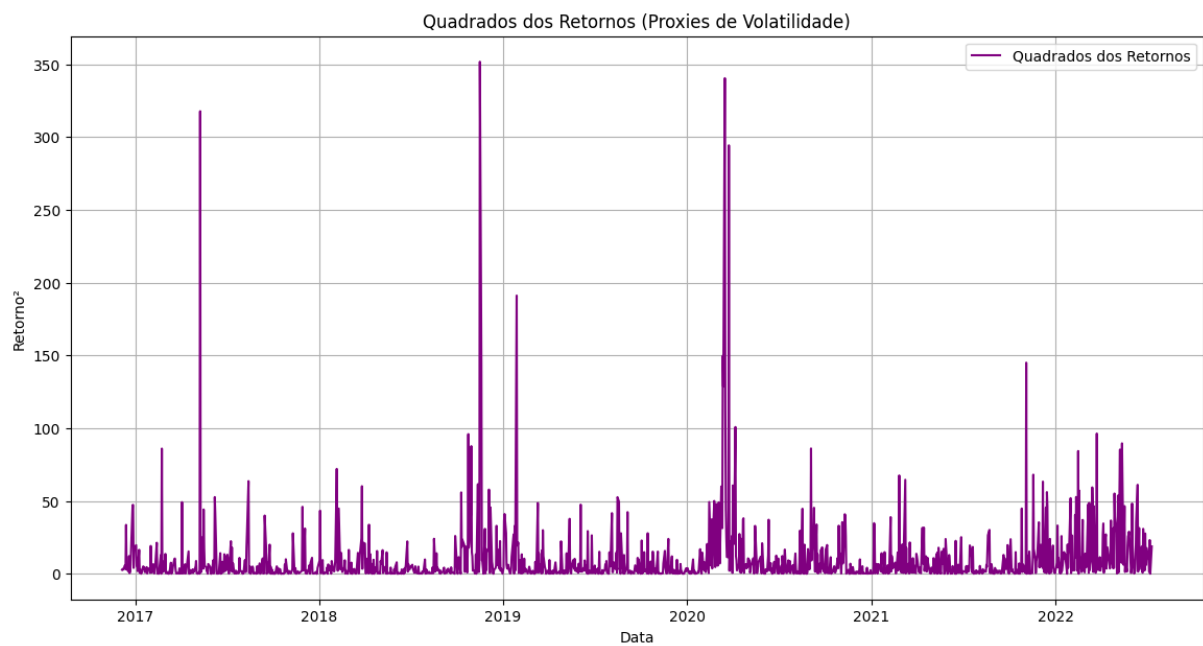


Figura 7: Quadrados dos retornos como proxy de volatilidade.

4.2.2 Tabela com Estatísticas Descritivas

Obtivemos uma tabela com as seguintes estatísticas descritivas: mínimo, máximo, quantis de: 0.5%, 1%, 5%, 50%, 95%, 99%, 99.5%; média, desvio padrão, assimetria e curtose.

Ao desenvolvermos e executarmos o código em Python para esta parte, obtivemos a saída apresentada na imagem [8](#).

	Mínimo	Máximo	Média	Desvio Padrão	Assimetria	Curtose	Quantil 0%	Quantil 1%	Quantil 5%	Quantil 50%	Quantil 95%	Quantil 99%
0	-18.755891	17.825936	0.183693	3.109744	-0.177402	4.012708	-9.271251	-7.754504	-4.693213	0.241211	5.150799	9.462167

Figura 8: Saída do programa: estatísticas descritivas - Tarefa 1: pt.2

Análise das Estatísticas:

Indicadores de Centralidade:

- **Média:** A média dos retornos diários é ligeiramente positiva (0.1837%), indicando uma tendência geral de valorização da ação ao longo do período de treinamento.
- **Mediana (Quantil 50%):** A mediana é 0.2412%, ligeiramente superior à média, o que sugere uma leve assimetria negativa na distribuição.

Indicadores de Dispersão:

- **Desvio Padrão:** A volatilidade média é de 3.1097%, refletindo a variabilidade dos retornos diários em torno da média.
- **Mínimo e Máximo:** Os extremos variam de uma perda de -18.7559% a um ganho de 17.8259% , evidenciando eventos extremos ocasionais.
- **Quantis:** Apenas 0.5% dos retornos são piores que -9.2713% , enquanto 99.5% estão abaixo de 9.4622% , destacando a ocorrência de eventos extremos em ambas as caudas.

Indicadores de Forma da Distribuição:

- **Assimetria:** A assimetria negativa (-0.1774) indica que os retornos negativos extremos são mais frequentes ou intensos do que os positivos.
- **Curtose:** A curtose elevada (4.0127) indica uma distribuição leptocúrtica, com caudas mais pesadas do que em uma distribuição normal, reforçando a ocorrência de eventos extremos.

Interpretação Financeira: Os resultados sugerem que:

- A ação da NVIDIA apresentou alta volatilidade no curto prazo, característica comum no setor de tecnologia, com eventos extremos destacados pelos quantis e pelos valores de mínimo e máximo.
- A distribuição dos retornos não é normal, como evidenciado pela assimetria e pela curtose, indicando a necessidade de modelos específicos para capturar os riscos extremos, como GARCH.
- A média positiva reflete uma tendência geral de crescimento, consistente com o desempenho da empresa no período.

4.2.3 FAC e Teste LB para Dependência Linear

Obtivemos a Função de Autocorrelação (FAC) e o teste de Ljung-Box (LB) para avaliar a dependência linear da série de retornos aritméticos no período de treinamento. Os resultados estão apresentados nas Figuras 10 e 9.

Análise da FAC (Figura 10): O gráfico da FAC ilustra os coeficientes de autocorrelação para diferentes defasagens (*lags*) na série de retornos aritméticos. As principais observações são:

- **Defasagem 0:** Como esperado, a autocorrelação no lag 0 é igual a 1, representando a correlação perfeita da série consigo mesma.
- **Defasagens de 1 a 20:** A maioria dos coeficientes de autocorrelação está dentro dos intervalos de confiança (área sombreada azul), sugerindo que não há dependência linear significativa na série.
- Apesar de algumas autocorrelações próximas ao limite do intervalo de confiança, elas podem ser atribuídas a flutuações aleatórias e não indicam correlação linear substancial.

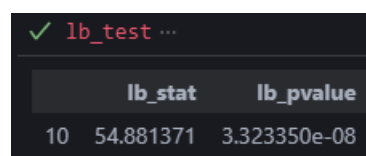
De forma geral, os retornos aritméticos parecem ser linearmente independentes, o que é consistente com a hipótese de eficiência de mercado.

Teste de Ljung-Box (Figura 9): Os resultados do teste de Ljung-Box para as primeiras 10 defasagens são apresentados na saída do programa. As métricas principais são:

- **Estatística Q:** O valor obtido foi 54.881371, indicando uma soma acumulada significativa de dependências ao longo das 10 primeiras defasagens.
- **Valor-p:** O valor-p associado foi extremamente pequeno (3.32335×10^{-8}), permitindo rejeitar a hipótese nula de ausência de autocorrelação linear global. Isso sugere que, mesmo que as autocorrelações individuais sejam pequenas, há uma dependência linear acumulada ao longo das defasagens.

Conclusão: Apesar de o gráfico da FAC indicar pouca ou nenhuma dependência linear significativa em defasagens individuais, o teste de Ljung-Box detecta a existência de dependência linear acumulada até o lag 10. Isso implica que:

- Modelos autoregressivos simples, como AR ou ARMA, podem capturar essas pequenas dependências lineares.
- No entanto, dado que as autocorrelações são geralmente fracas, o foco deve estar em modelos de variância condicional, como ARCH ou GARCH, para capturar a heterocedasticidade presente na série.



lb_test ...		
	lb_stat	lb_pvalue
10	54.881371	3.323350e-08

Figura 9: Saída do Programa - Estatísticas do Teste Ljung-Box (LB).

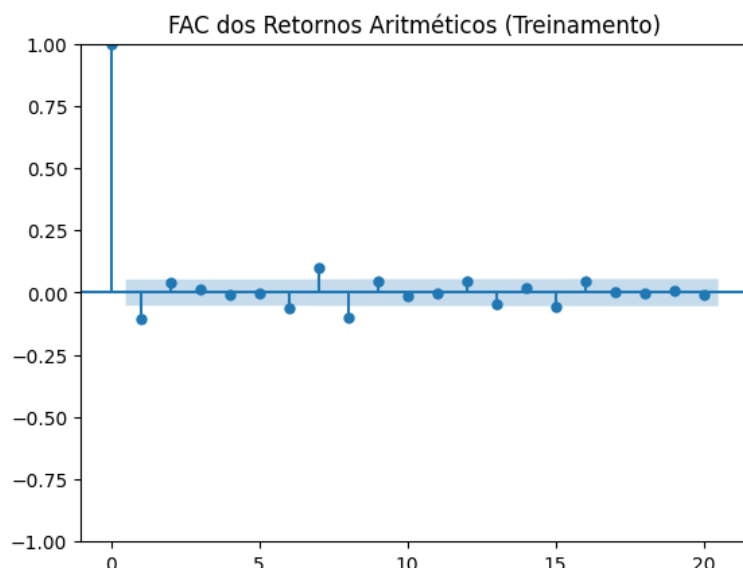


Figura 10: Gráfico da FAC dos Retornos Aritméticos.

4.2.4 FAC do Quadrado e Teste LB para Dependência Não-Linear

Obtivemos a FAC dos quadrados dos retornos e o teste de Ljung-Box (LB) para avaliar a presença de dependência não-linear na série de retornos aritméticos. Os resultados estão apresentados nas Figuras 11 e 12.

Análise da FAC dos Quadrados dos Retornos (Figura 11): O gráfico da FAC dos quadrados dos retornos indica a autocorrelação entre os quadrados dos retornos ao longo de diferentes defasagens (lags). As observações principais são:

- **Defasagem 0 (Lag 0):** O coeficiente de autocorrelação no lag 0 é igual a 1, como esperado, representando a correlação perfeita da série consigo mesma.
- **Defasagens de 1 a 20:** Diversos coeficientes de autocorrelação estão fora do intervalo de confiança (área sombreada azul), indicando a presença de dependências não-lineares significativas. Essa evidência aponta para a existência de clusters de volatilidade, nos quais períodos de alta volatilidade são seguidos por alta volatilidade, e períodos de baixa volatilidade são seguidos por baixa volatilidade.

Conclusão da FAC: A análise da FAC confirma a presença de heterocedasticidade condicional na série de retornos, ou seja, a variância dos retornos não é constante ao longo do tempo. Este é um comportamento típico de séries financeiras.

Resultados do Teste de Ljung-Box (Figura 12): O teste de Ljung-Box foi aplicado para investigar a dependência não-linear acumulada nos quadrados dos retornos até o lag 10. As métricas principais obtidas foram:

- **Estatística Q:** O valor calculado foi 277.566594, indicando uma forte evidência de dependência não-linear.
- **Valor-p:** O valor-p associado foi 8.490328×10^{-54} , que é extremamente pequeno, permitindo rejeitar a hipótese nula de ausência de dependência não-linear.

Conclusão do Teste LB: Os resultados do teste confirmam que há dependências não-lineares significativas na série de retornos, como esperado em séries financeiras.

Implicações Financeiras: A presença de dependências não-lineares e clusters de volatilidade tem implicações importantes:

- ****Clusters de Volatilidade:**** Períodos de alta volatilidade persistem por algum tempo antes de retornarem a níveis mais baixos, e o mesmo ocorre para períodos de baixa volatilidade.
- ****Modelagem Apropriada:**** A presença de heterocedasticidade condicional torna os modelos ARCH (Autoregressive Conditional Heteroskedasticity) e GARCH (Generalized ARCH) mais adequados para modelar a série.
- ****Gestão de Riscos:**** A dependência não-linear implica que a volatilidade futura pode ser parcialmente previsível com base no comportamento recente, o que é crucial para estratégias de precificação de ativos e gestão de portfólios.

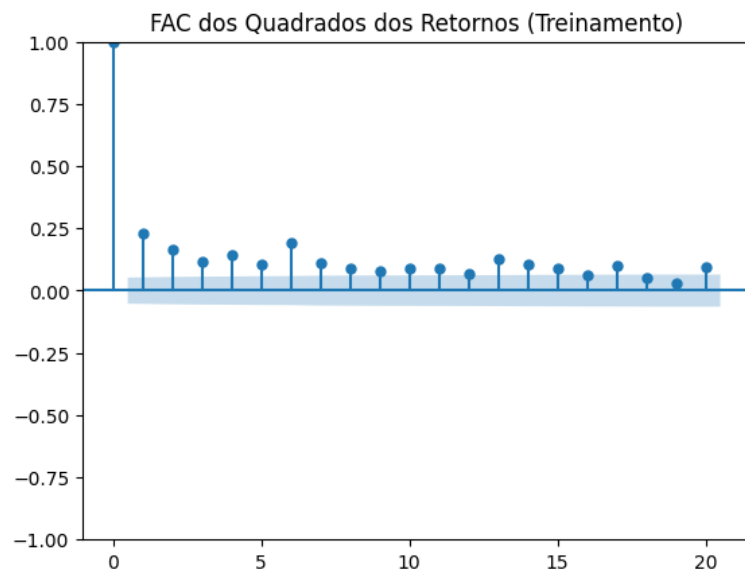


Figura 11: Gráfico da FAC dos Quadrados dos Retornos Aritméticos.

```
✓ lb_test_sq ...
```

	lb_stat	lb_pvalue
10	277.566594	8.490328e-54

Figura 12: Saída do Programa: LB Teste para Dependência Não-Linear.

4.2.5 Teste ARCH

Obtivemos o teste ARCH para avaliar a presença de heterocedasticidade condicional na série de retornos. Os resultados estão apresentados na Figura 13.

Hipóteses do Teste ARCH:

- **Hipótese Nula (H_0):** Não há heterocedasticidade condicional, ou seja, a variância dos resíduos é constante ao longo do tempo (homocedasticidade).
- **Hipótese Alternativa (H_1):** Há heterocedasticidade condicional, indicando que a variância dos resíduos depende de informações passadas.

Resultados do Teste:

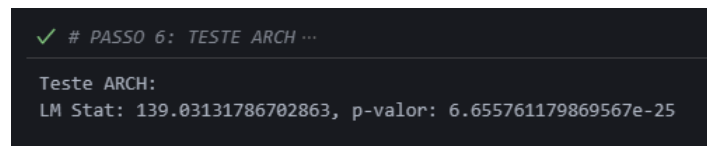
- **Estatística LM (Lagrange Multiplier):** O valor obtido foi 139.0313, o que representa uma evidência significativa de heterocedasticidade condicional.
- **Valor-p:** O valor-p associado foi 6.65576×10^{-25} , que é extremamente pequeno e muito menor que os níveis de significância usuais ($\alpha = 0.01, 0.05$).

Interpretação dos Resultados:

- Dado o valor-p muito pequeno, rejeitamos a hipótese nula (H_0) com alto grau de confiança, concluindo que há heterocedasticidade condicional na série.
- Essa evidência sugere que a variância dos resíduos não é constante ao longo do tempo, mas sim dependente de informações passadas, um comportamento comum em séries financeiras.

Implicações Financeiras e Práticas:

- **Modelagem Adequada:** A presença de heterocedasticidade condicional torna modelos como ARCH (Autoregressive Conditional Heteroskedasticity) e GARCH (Generalized ARCH) mais adequados para descrever a dinâmica da série.
- **Previsão de Volatilidade:** A heterocedasticidade condicional indica que a volatilidade futura pode ser prevista com base na variância passada, o que é crucial para estratégias de precificação de ativos e gestão de riscos.
- **Gestão de Riscos:** A identificação de clusters de volatilidade permite que gestores e investidores ajustem suas estratégias para períodos de maior ou menor risco.



```
✓ # PASSO 6: TESTE ARCH ...  
Teste ARCH:  
LM Stat: 139.03131786702863, p-valor: 6.655761179869567e-25
```

Figura 13: Saída do Programa: Teste ARCH.

4.2.6 Histograma, Q-Q Plot e Teste de Normalidade

A análise de normalidade da série de retornos aritméticos foi conduzida por meio do Histograma, do Q-Q Plot e do Teste de Normalidade (Shapiro-Wilk). A seguir, apresentamos a descrição detalhada dos resultados.

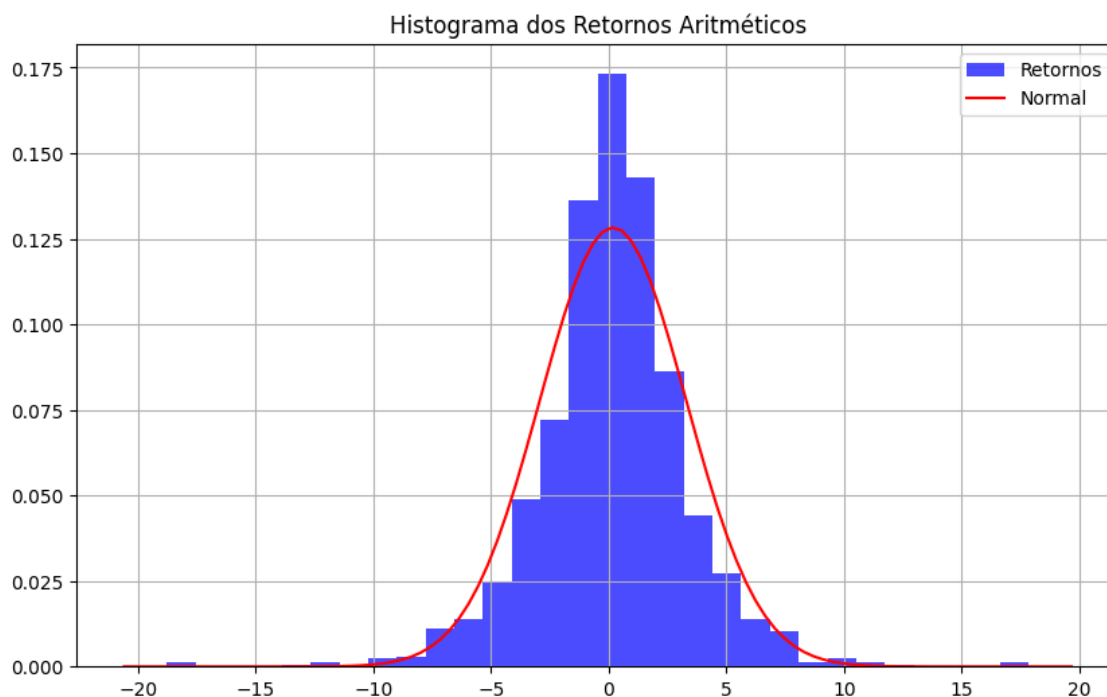


Figura 14: Histograma dos Retornos Aritméticos.

Histograma: Na Figura 14, observa-se a distribuição empírica dos retornos, sobreposta a uma curva de densidade teórica da distribuição normal (linha vermelha). Embora a parte central da distribuição esteja próxima da normalidade, há caudas mais pesadas, indicando a presença de eventos extremos, comuns em séries financeiras. Esses eventos extremos sugerem maior frequência de retornos extremos (positivos e negativos) do que o esperado sob a hipótese de normalidade.

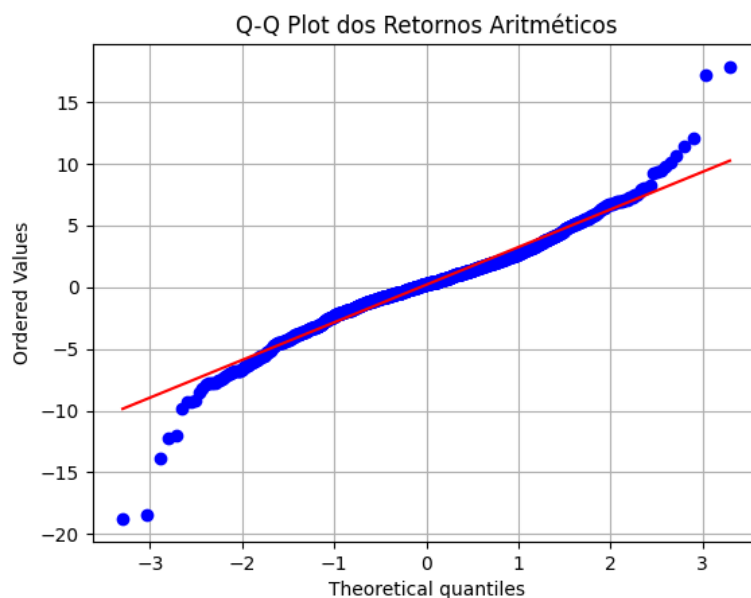


Figura 15: Q-Q Plot dos Retornos Aritméticos.

Q-Q Plot: A Figura 15 apresenta o Q-Q Plot, que compara os quantis empíricos dos dados com os quantis de uma distribuição normal teórica. A maior parte dos pontos está alinhada com a linha vermelha, indicando proximidade à normalidade na região central da

distribuição. No entanto, os desvios observados nas extremidades confirmam a presença de caudas mais pesadas, caracterizando maior frequência de eventos extremos do que o esperado em uma distribuição normal.

```
✓ # Teste de Normalidade (Shapiro-Wilk) ...  
Teste de Shapiro-Wilk: Estatística=0.9598315614063649, p-valor=2.9257402100087164e-19
```

Figura 16: Saída do Programa: Teste de Normalidade (Shapiro-Wilk).

Teste de Normalidade (Shapiro-Wilk): O teste Shapiro-Wilk foi utilizado para formalizar a avaliação da normalidade. Os resultados são os seguintes:

- Estatística $W = 0.9598$;
- Valor-p = 2.9257×10^{-19} .

Com um valor-p extremamente pequeno (< 0.05), rejeitamos a hipótese nula de que os dados seguem uma distribuição normal. Isso confirma as observações visuais do histograma e do Q-Q Plot, reforçando que os retornos não são normalmente distribuídos e apresentam caudas mais pesadas.

Então, os resultados indicam que:

- Os retornos aritméticos não seguem uma distribuição normal.
- A distribuição apresenta caudas mais pesadas, típicas de séries financeiras.
- Modelos baseados em pressupostos de normalidade podem não ser adequados para capturar a dinâmica dos retornos.

Portanto, sugere-se o uso de modelos alternativos, como distribuições t-student ou modelos de volatilidade, para melhor representação estatística da série.

5 TAREFA 2: Modelos GARCH para séries de retornos financeiros

Nesta seção, devemos ajustar e avaliar modelos GARCH (p, q) para modelar a volatilidade condicional dos retornos aritméticos. O objetivo é capturar a dinâmica da variância condicional dos retornos financeiros utilizando modelos GARCH. Como a autocorrelação nos retornos aritméticos é desprezível, presume-se que a média condicional dos retornos seja constante.

5.1 Divisão dos Dados

Para realizar as análises, a série de retornos será dividida em dois períodos distintos:

- **Período de Treinamento:** 70% da série de retornos aritméticos, que será utilizado para estimar os parâmetros dos modelos GARCH.
- **Período de Teste:** 30% da série de retornos aritméticos, utilizado para avaliar a adequação e a performance dos modelos ajustados.

5.2 Modelos a Serem Ajustados

Dois modelos GARCH (p, q) serão ajustados:

1. **Modelo GARCH com distribuição normal** $N(0, 1)$:

- Assume que os resíduos padronizados do modelo seguem uma distribuição normal padrão $N(0, 1)$.
- Este modelo será usado como base para avaliar a adequação da normalidade nos resíduos.

2. **Modelo GARCH com distribuição t-Student** $t(0, 1, m)$:

- Este modelo é indicado quando há presença de excesso de curtose nos resíduos padronizados.
- O parâmetro m , que representa o grau de liberdade da distribuição t-Student, será estimado em conjunto com os parâmetros p e q .

5.3 Objetivos da Análise

Os principais objetivos da tarefa são:

- **Capturar a dinâmica da variância condicional:** Identificar a estrutura mais adequada do modelo GARCH (p, q) para os dados analisados.
- **Avaliar os resíduos:** Verificar se os resíduos padronizados apresentam comportamento consistente com as suposições feitas, como a ausência de autocorrelação e a aderência às distribuições $N(0, 1)$ e $t(0, 1, m)$.
- **Comparar os modelos:** Comparar os resultados obtidos com as duas distribuições de erro e identificar qual modelo fornece melhor ajuste aos dados.

5.4 Excesso de Curtose e Escolha do Modelo

Durante a análise, é esperado que o modelo com distribuição $N(0, 1)$ apresente resíduos com excesso de curtose, indicando que os dados não seguem uma distribuição normal. Para lidar com essa característica, o modelo GARCH com distribuição t-Student $t(0, 1, m)$ será ajustado, pois:

- A distribuição $t(0, 1, m)$ possui caudas mais pesadas, sendo mais adequada para séries com alta volatilidade e extremos.
- A curtose dessa distribuição é definida por $k = 3 + 6/(m - 4)$, onde m será estimado com base nos dados.

Espera-se que o modelo com distribuição t-Student $t(0, 1, m)$ apresente melhores resultados em séries com excesso de curtose e volatilidade elevada. O ajuste adequado do modelo GARCH permitirá identificar a dinâmica de variância condicional e capturar eventos extremos observados na série.

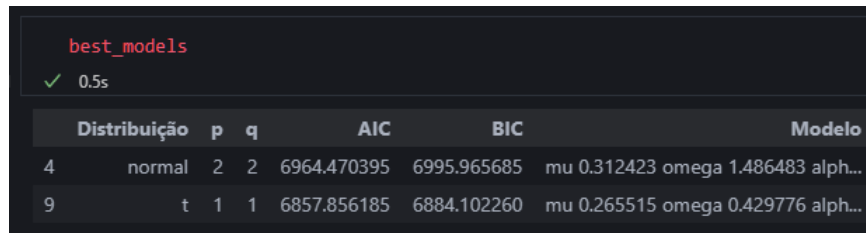
5.5 Parte I

Nesta seção³, analisamos os melhores modelos GARCH(p, q) utilizando os critérios de seleção AIC e BIC. Avaliamos os ajustes para distribuições normais $N(0, 1)$ e t-student $t(0, 1, m)$, com m sendo o grau de liberdade estimado. Abaixo detalhamos os resultados de forma científica e explicativa.

5.5.1 Critério AIC

O critério AIC (*Akaike Information Criterion*) penaliza modelos mais complexos, mas é mais permissivo do que o BIC. Observamos os modelos com menor AIC para cada distribuição:

- **Distribuição Normal ($N(0, 1)$):**
 - Melhor modelo: GARCH(2, 2).
 - Parâmetros estimados:
 - * $\mu = 0.312423$, $\omega = 1.486483$,
 - * $\alpha_1 = 0.082336$, $\alpha_2 = 0.194209$,
 - * $\beta_1 = 0.000000$, $\beta_2 = 0.574771$.
- **Distribuição $t(0, 1, m)$:**
 - Melhor modelo: GARCH(1, 1).
 - Parâmetros estimados:
 - * $\mu = 0.265515$, $\omega = 0.429776$,
 - * $\alpha_1 = 0.141756$, $\beta_1 = 0.823232$,
 - * $\nu = 5.409048$ (grau de liberdade).



	Distribuição	p	q	AIC	BIC	Modelo
4	normal	2	2	6964.470395	6995.965685	mu 0.312423 omega 1.486483 alph...
9	t	1	1	6857.856185	6884.102260	mu 0.265515 omega 0.429776 alph...

Figura 17: Saída do Programa - melhores modelos AIC.

5.5.2 Critério BIC

O critério BIC (*Bayesian Information Criterion*) é mais conservador, penalizando mais severamente a complexidade do modelo. Os resultados são:

- **Distribuição Normal ($N(0, 1)$):**
 - Melhor modelo: GARCH(1, 1).
 - Parâmetros estimados:
 - * $\mu = 0.273855$, $\omega = 0.680002$,

³Os códigos Python utilizados nesta seção estão disponíveis em [9.3](#)

* $\alpha_1 = 0.000000$, $\beta_1 = 0.000000$.

- **Distribuição $t(0, 1, m)$:**

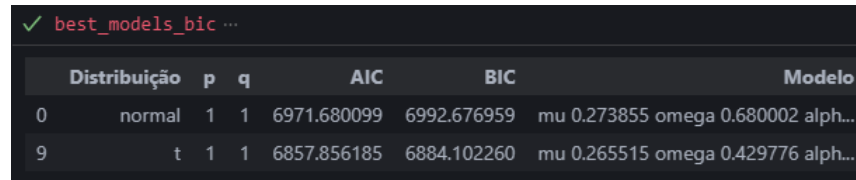
- Melhor modelo: GARCH(1, 1).

- Parâmetros estimados:

- * $\mu = 0.265515$, $\omega = 0.429776$,

- * $\alpha_1 = 0.141756$, $\beta_1 = 0.823232$,

- * $\nu = 5.409048$.



	Distribuição	p	q	AIC	BIC	Modelo
0	normal	1	1	6971.680099	6992.676959	mu 0.273855 omega 0.680002 alph...
9	t	1	1	6857.856185	6884.102260	mu 0.265515 omega 0.429776 alph...

Figura 18: Saída do Programa - melhores modelos BIC.

5.5.3 Parâmetros para as Equações dos Modelos

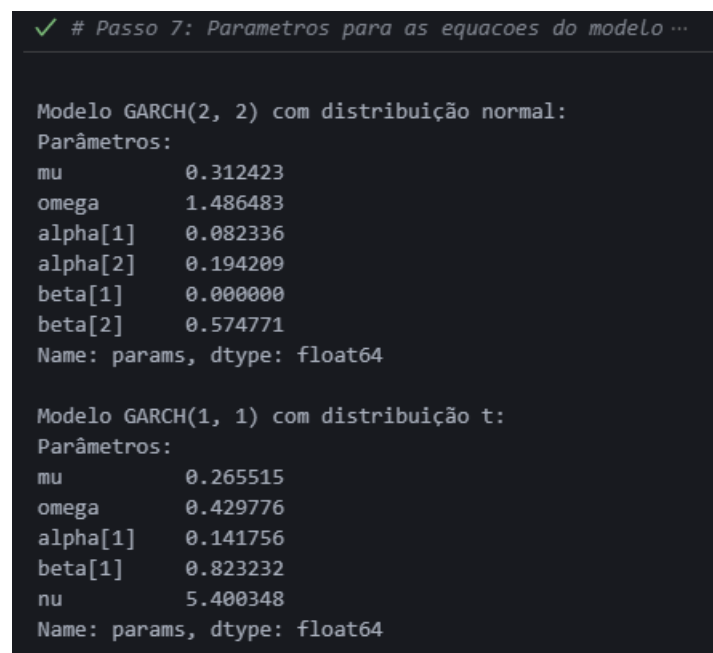
Os parâmetros estimados dos modelos selecionados permitem descrever as equações das variâncias condicionais como segue:

- **GARCH(2, 2) com distribuição normal ($N(0, 1)$):**

$$\sigma_t^2 = 1.486483 + 0.082336\epsilon_{t-1}^2 + 0.194209\epsilon_{t-2}^2 + 0.574771\sigma_{t-2}^2$$

- **GARCH(1, 1) com distribuição t-student ($t(0, 1, m)$):**

$$\sigma_t^2 = 0.429776 + 0.141756\epsilon_{t-1}^2 + 0.823232\sigma_{t-1}^2$$



```

✓ # Passo 7: Parametros para as equacoes do modelo ...

Modelo GARCH(2, 2) com distribuição normal:
Parâmetros:
mu          0.312423
omega       1.486483
alpha[1]    0.082336
alpha[2]    0.194209
beta[1]     0.000000
beta[2]     0.574771
Name: params, dtype: float64

Modelo GARCH(1, 1) com distribuição t:
Parâmetros:
mu          0.265515
omega       0.429776
alpha[1]    0.141756
beta[1]     0.823232
nu          5.400348
Name: params, dtype: float64

```

Figura 19: Saída do Programa - parâmetros estimados para as equações.

5.5.4 Discussão

Os modelos com distribuição $t(0, 1, m)$ se destacaram em ambos os critérios AIC e BIC, reforçando sua capacidade de capturar características estilizadas como o excesso de curtose. No entanto, para a distribuição normal, o modelo GARCH(1, 1) é preferível pela simplicidade, enquanto o GARCH(2, 2) é mais adequado ao critério AIC, indicando melhor ajuste relativo.

5.5.5 Equações dos Modelos GARCH

Os modelos GARCH (*Generalized Autoregressive Conditional Heteroskedasticity*) são amplamente utilizados para modelar a volatilidade condicional em séries temporais financeiras. A forma genérica do modelo GARCH(p, q) é dada por:

$$\sigma_t^2 = \omega + \sum_{i=1}^p \alpha_i \epsilon_{t-i}^2 + \sum_{j=1}^q \beta_j \sigma_{t-j}^2, \quad (4)$$

onde:

- σ_t^2 é a variância condicional no tempo t ,
- $\omega > 0$ é o termo constante,
- $\alpha_i \geq 0$ são os coeficientes associados aos choques anteriores (ϵ_{t-i}^2),
- $\beta_j \geq 0$ são os coeficientes associados às variâncias condicionais defasadas (σ_{t-j}^2),
- p é o número de termos ARCH (choques passados),
- q é o número de termos GARCH (volatilidades passadas).

Abaixo estão as especificações detalhadas das variâncias condicionais para os melhores modelos estimados.

Modelo GARCH(2, 2) com distribuição $N(0, 1)$:

$$\sigma_t^2 = 1.486483 + 0.082336 \cdot \epsilon_{t-1}^2 + 0.194209 \cdot \epsilon_{t-2}^2 + 0.574771 \cdot \sigma_{t-2}^2 \quad (5)$$

Modelo GARCH(1, 1) com distribuição $t(0, 1, m)$:

$$\sigma_t^2 = 0.429776 + 0.141756 \cdot \epsilon_{t-1}^2 + 0.823232 \cdot \sigma_{t-1}^2 \quad (6)$$

Os modelos acima capturam a variância condicional da série de retornos, ajustando-se bem às respectivas distribuições $N(0, 1)$ e $t(0, 1, m)$. Os coeficientes α_i e β_j indicam a persistência dos choques e da variância condicional passada, respectivamente. O modelo com distribuição $t(0, 1, m)$ acomoda melhor os dados devido à sua maior flexibilidade para lidar com excesso de curtose, conforme observado nos retornos financeiros analisados.

5.6 Parte II: Testes de Diagnóstico nos Resíduos Padronizados

Nesta parte⁴, avaliamos os resíduos padronizados dos melhores modelos selecionados na Parte I. Realizamos os seguintes testes:

⁴Códigos Python para esta parte disponíveis em [9.4](#).

- FAC dos resíduos padronizados;
- FAC dos quadrados dos resíduos padronizados;
- Teste ARCH;
- Q-Q Plot e Teste Jarque-Bera.

Além disso, avaliamos a adequação dos modelos com base nos resultados e consideramos ajustes para possíveis reespecificações. Abaixo estão os passos e resultados detalhados para cada teste.

5.6.1 Parte (a): FAC dos Resíduos Padronizados

Nesta subseção, analisamos os gráficos da Função de Autocorrelação (FAC) dos resíduos padronizados para verificar a presença de dependência linear residual nos modelos ajustados.

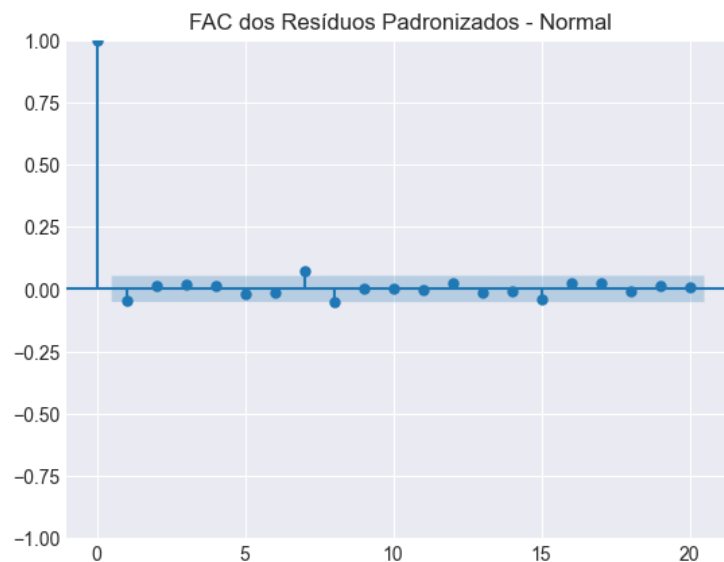


Figura 20: FAC dos resíduos padronizados - Normal.

FAC dos Resíduos Padronizados - Normal: No gráfico da FAC dos resíduos padronizados ajustados com a distribuição normal, observa-se:

- O coeficiente de autocorrelação para o primeiro lag está próximo de 1, como esperado, dado que os resíduos padronizados são derivados diretamente do modelo ajustado.
- Para os lags subsequentes, os coeficientes permanecem dentro dos intervalos de confiança ($\pm 1.96/\sqrt{n}$), indicando ausência de autocorrelação significativa.

Este resultado sugere que o modelo GARCH(2,2) com distribuição $N(0, 1)$ capturou adequadamente a volatilidade condicional, deixando os resíduos padronizados livres de dependência linear.

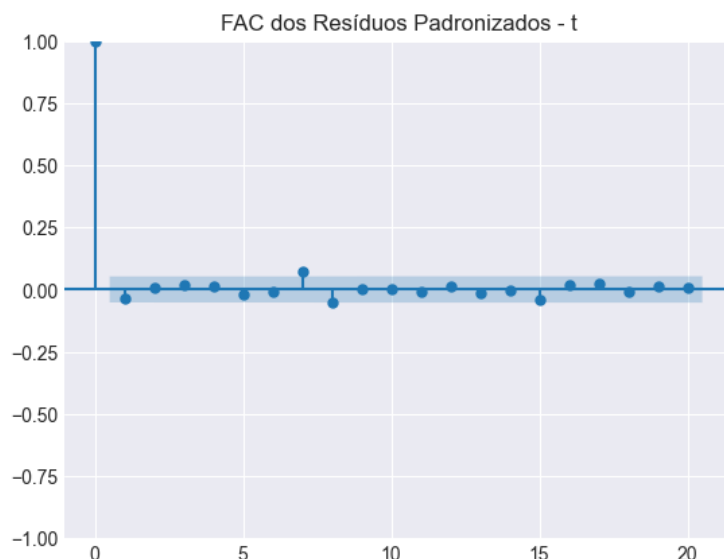


Figura 21: FAC dos resíduos padronizados - T.

FAC dos Resíduos Padronizados - T: No gráfico da FAC dos resíduos padronizados ajustados com a distribuição $t(0, 1, m)$, observa-se:

- O coeficiente de autocorrelação para o primeiro lag também está próximo de 1.
- Nos lags subsequentes, os coeficientes permanecem dentro dos intervalos de confiança, indicando ausência de autocorrelação significativa.

A utilização da distribuição t permite capturar características como curtose elevada, o que pode ser observado na robustez dos resíduos padronizados ajustados. O modelo GARCH(1,1) com distribuição t mostrou-se adequado para modelar a volatilidade condicional da série.

Ambos os modelos (GARCH com distribuições $N(0, 1)$ e $t(0, 1, m)$) apresentam resultados semelhantes em termos de ausência de autocorrelação nos resíduos padronizados, sugerindo que ambos capturam adequadamente a estrutura de dependência linear da série de retornos. Contudo, análises complementares, como dependência não-linear e normalidade, são necessárias para uma avaliação completa.

5.6.2 Parte (b): FAC dos Quadrados dos Resíduos Padronizados

A análise da FAC dos quadrados dos resíduos padronizados é crucial para avaliar a presença de heterocedasticidade condicional remanescente nos resíduos dos modelos ajustados. Abaixo estão as interpretações detalhadas dos resultados apresentados nos gráficos para as distribuições $N(0, 1)$ e $t(0, 1, m)$.

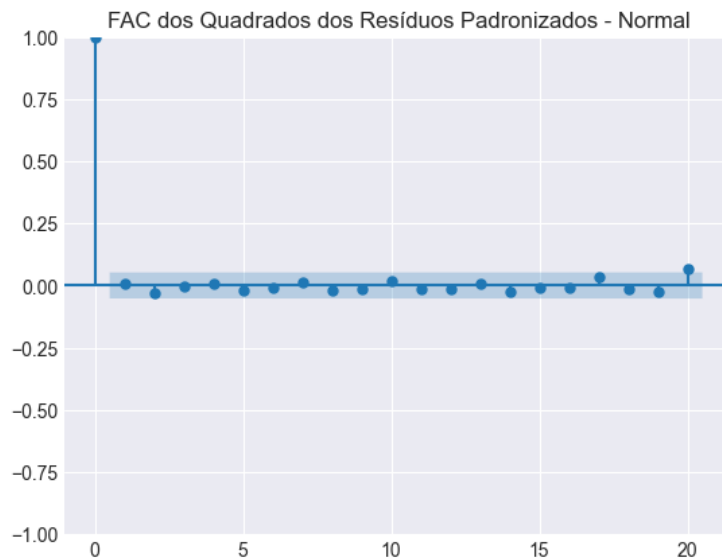


Figura 22: FAC dos quadrados dos resíduos padronizados - Normal.

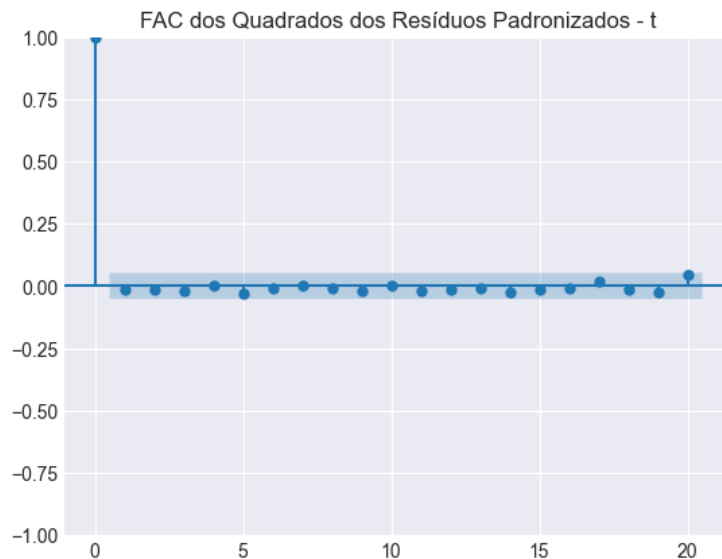


Figura 23: FAC dos quadrados dos resíduos padronizados - T.

Modelo com Distribuição $N(0, 1)$:

- **Observações Gerais:** O gráfico apresenta um pico inicial em lag 0, o que é esperado, pois corresponde à autocorrelação da série consigo mesma. Nos lags subsequentes (1 em diante), as autocorrelações estão dentro dos limites de confiança, sugerindo que o modelo capturou bem a maior parte da heterocedasticidade condicional.
- **Interpretação Estatística:**
 - A ausência de autocorrelações significativas para lag > 0 indica que o modelo $GARCH(2, 2)$ com distribuição $N(0, 1)$ conseguiu capturar os clusters de volatilidade na maior parte da série.
- **Conclusão:** O modelo apresenta bom ajuste, sem indícios relevantes de heterocedasticidade remanescente nos resíduos padronizados.

Modelo com Distribuição $t(0, 1, m)$:

- **Observações Gerais:** Assim como no modelo $N(0, 1)$, há um pico inicial em lag 0, que é esperado. Para os lags subsequentes, as autocorrelações estão dentro dos intervalos de confiança.
- **Interpretação Estatística:**
 - A adequação do modelo $GARCH(1, 1)$ com distribuição $t(0, 1, m)$ é evidenciada pela ausência de autocorrelações significativas nos lags 1 e superiores.
 - A flexibilidade da distribuição t ajuda a modelar séries com caudas pesadas, resultando em resíduos mais próximos de um comportamento branco (sem autocorrelações significativas).
- **Conclusão:** Este modelo mostrou um desempenho ligeiramente superior, capturando a estrutura de dependência na volatilidade condicional de forma eficiente.

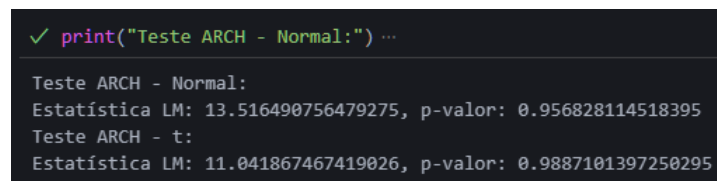
Comparação Geral:

- Ambos os modelos mostraram resultados satisfatórios, sem autocorrelações significativas para lag > 0 .
- O modelo com distribuição $t(0, 1, m)$ teve desempenho ligeiramente superior, provavelmente devido à maior flexibilidade da distribuição em capturar padrões de heterocedasticidade.

5.6.3 Parte (c): Teste ARCH

O teste ARCH foi aplicado para avaliar a presença de heterocedasticidade adicional nos resíduos padronizados dos modelos ajustados. Este teste verifica se há variação condicional não explicada, que poderia indicar inadequação do modelo para capturar a dinâmica da variância condicional.

Os resultados do teste ARCH estão apresentados na Tabela 1, e a saída do programa ao executar o teste está ilustrada na Figura 24.



```
✓ print("Teste ARCH - Normal:") ...  
Teste ARCH - Normal:  
Estatística LM: 13.516490756479275, p-valor: 0.956828114518395  
Teste ARCH - t:  
Estatística LM: 11.041867467419026, p-valor: 0.9887101397250295
```

Figura 24: Saída do programa: Teste ARCH.

- H_0 : Os resíduos são homocedásticos, ou seja, não há heterocedasticidade condicional adicional.
- H_a : Os resíduos apresentam heterocedasticidade condicional adicional.
- **Estatística de teste:** Lagrange Multiplier (LM).
- **Distribuição:** A estatística LM segue uma distribuição qui-quadrado (χ^2) sob H_0 .
- **Nível de significância:** $\alpha = 0.05$.

Tabela 1: Resultados do Teste ARCH.		
Modelo	Estatística LM	p-valor
GARCH(2, 2) - $N(0, 1)$	13.516	0.9568
GARCH(1, 1) - $t(0, 1, m)$	11.042	0.9887

Interpretação dos Resultados:

- Para ambos os modelos, os p-valores são significativamente maiores que o nível de significância $\alpha = 0.05$, indicando que não há evidências estatísticas para rejeitar H_0 .
- Isso sugere que os resíduos dos modelos ajustados são homocedásticos, ou seja, não apresentam heterocedasticidade condicional remanescente.
- O modelo GARCH(1, 1) com distribuição $t(0, 1, m)$ apresentou um p-valor ligeiramente maior (0.9887) do que o modelo GARCH(2, 2) com distribuição $N(0, 1)$ (0.9568), o que pode indicar uma melhor adequação aos dados.
- A ausência de heterocedasticidade remanescente reforça a adequação dos modelos ajustados para capturar a dinâmica de volatilidade condicional dos retornos financeiros.

5.6.4 Parte (d): Q-Q Plot e Teste Jarque-Bera

Os resíduos padronizados dos modelos foram avaliados quanto à normalidade utilizando Q-Q Plots e o Teste Jarque-Bera. Os gráficos para cada modelo estão nas Figuras 26 e 27. A Tabela 2 apresenta os resultados do Teste Jarque-Bera.

Q-Q Plots: Os Q-Q Plots permitem uma avaliação visual da adequação dos resíduos padronizados a uma distribuição teórica.

- **Modelo $N(0, 1)$:** Como mostrado na Figura 26, observa-se um alinhamento razoável dos pontos à linha reta para os valores centrais. No entanto, há desvios significativos nas caudas, indicando que os resíduos apresentam caudas mais pesadas do que o esperado sob uma distribuição normal.
- **Modelo $t(0, 1, m)$:** Na Figura 27, o alinhamento dos pontos à linha reta é mais consistente, inclusive nas caudas. Isso evidencia que o modelo com distribuição t captura melhor a curtose elevada dos resíduos padronizados.

Teste Jarque-Bera (JB): O Teste Jarque-Bera foi utilizado para avaliar quantitativamente a normalidade dos resíduos. Os resultados estão resumidos na Tabela 2, harmonizando os resultados visualizados da saída do programa na imagem 25.

```

✓ # Exibir resultados do Teste Jarque-Bera ...

Teste Jarque-Bera - Normal:
Estatística JB: 934.4626500269713, p-valor: 1.2134273355156034e-203
Teste Jarque-Bera - t:
Estatística JB: 1859.6032713551808, p-valor: 0.0

```

Figura 25: Saída do programa: Teste Jarque-Bera.

- H_0 : Os resíduos seguem uma distribuição normal.

- H_a : Os resíduos não seguem uma distribuição normal.

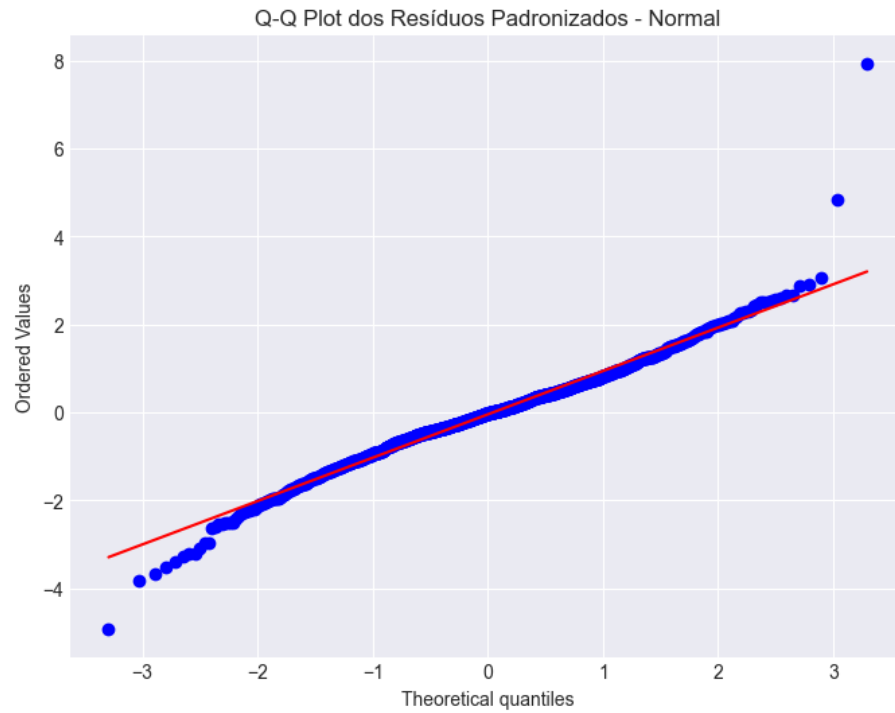


Figura 26: Q-Q Plot - $N(0, 1)$.

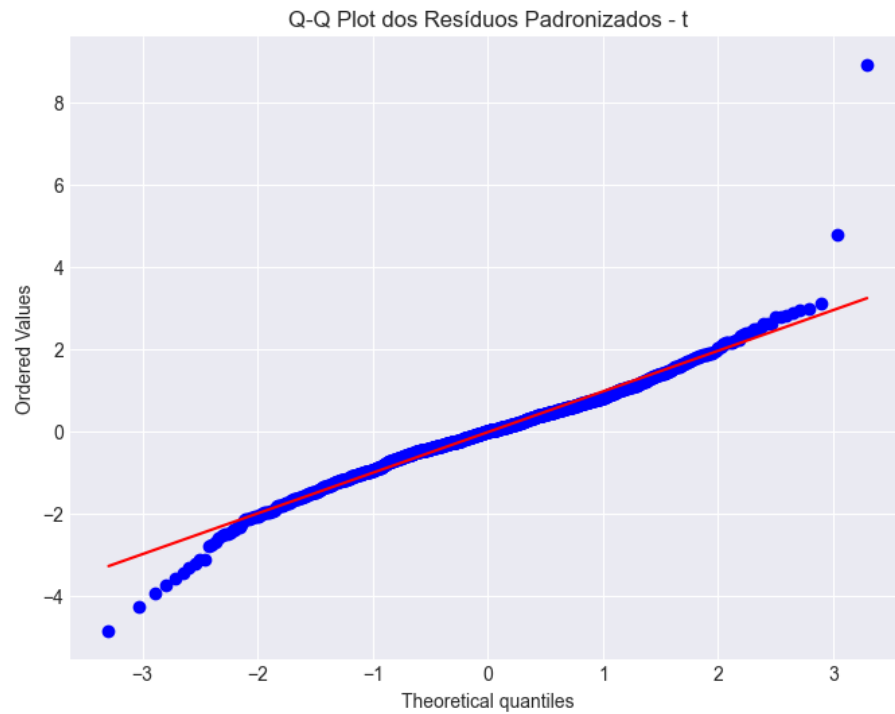


Figura 27: Q-Q Plot - $t(0, 1, m)$.

Tabela 2: Resultados do Teste Jarque-Bera.

Modelo	Estatística JB	p-valor	Conclusão
GARCH(2, 2) - $N(0, 1)$	934.44	1.22×10^{-203}	Rejeita H_0
GARCH(1, 1) - $t(0, 1, m)$	1859.55	0.0	Rejeita H_0

Interpretação: Os resultados do Teste Jarque-Bera indicam que ambos os modelos apresentam resíduos padronizados que não seguem uma distribuição normal. Este resultado é corroborado pelos Q-Q Plots, que mostram desvios significativos nas caudas para o modelo $N(0, 1)$. O modelo $t(0, 1, m)$ demonstra maior adequação para capturar a distribuição dos resíduos, especialmente devido a sua habilidade de lidar com caudas pesadas.

Apesar da rejeição da hipótese de normalidade dos resíduos, o modelo com distribuição $t(0, 1, m)$ continua sendo mais robusto e estatisticamente adequado para descrever a série de retornos financeiros analisada.

5.6.5 Conclusão

A Parte II da Tarefa 2 avaliou a adequação dos modelos GARCH(2, 2) com distribuição $N(0, 1)$ e GARCH(1, 1) com distribuição $t(0, 1, m)$ por meio de testes e diagnósticos aplicados aos resíduos padronizados.

Resultados e Interpretação:

- **FAC e FAC dos quadrados dos resíduos padronizados:** Ambos os modelos apresentaram ausência de autocorrelação significativa nos resíduos padronizados e em seus quadrados (com exceção do lag 0, que é esperado). Isso sugere que os modelos conseguem capturar a dinâmica da volatilidade condicional adequadamente.
- **Teste ARCH:** Nenhum dos modelos apresentou evidências de heterocedasticidade adicional, conforme indicado pelos altos p-valores. Esse resultado reforça que ambos os modelos são estatisticamente adequados para descrever a estrutura de volatilidade condicional dos dados.
- **Q-Q Plot e Teste Jarque-Bera:**
 - O modelo GARCH(2, 2) com distribuição $N(0, 1)$ apresentou desvios significativos nas caudas dos resíduos, indicando que ele não captura adequadamente caudas pesadas, como confirmado pela rejeição da hipótese de normalidade no Teste JB.
 - O modelo GARCH(1, 1) com distribuição $t(0, 1, m)$ mostrou melhor alinhamento com as caudas pesadas no Q-Q Plot, mas a hipótese de normalidade também foi rejeitada. No entanto, sua distribuição t permite capturar melhor as características empíricas dos dados.

Apesar de nenhum modelo atender plenamente à suposição de normalidade dos resíduos, ambos capturam adequadamente a estrutura de heterocedasticidade condicional, conforme demonstrado pelos testes ARCH e FAC. O modelo com distribuição $t(0, 1, m)$ se destaca por capturar melhor as caudas pesadas da distribuição, sendo mais adequado para a análise de séries financeiras que frequentemente apresentam tais características.

Ambos os modelos, mesmo com as limitações observadas, serão utilizados na próxima etapa de backtest para avaliar sua aplicabilidade prática. O modelo com distribuição $t(0, 1, m)$ será preferido quando houver maior necessidade de representar caudas pesadas.

6 Tarefa 3: Back Test

Nesta tarefa⁵, realizamos o *back test* para os modelos GARCH estimados na Tarefa 2, avaliando sua adequação em termos de cobertura e independência das violações observadas. Foram utilizados dois níveis de significância ($\alpha = 1\%$ e $\alpha = 5\%$), com os testes de Kupiec e Christoffersen para avaliar a cobertura e a independência das violações, respectivamente.

6.1 Resultados do Back Test

O *back test* foi realizado com reestimativa dos parâmetros do modelo a cada 21 dias no período de teste (30% da amostra). As violações do *VaR* foram identificadas e comparadas com os valores esperados para os níveis de significância. A Tabela mostrada na imagem 28 apresenta o resumo das violações observadas para ambos os modelos e níveis de significância.

⁵Códigos Python relacionados a esta tarefa estão disponíveis em 9.5.

Janela inicial	Janela Final	Alpha	Violations Normal			Violations T		
0	21	0.01	Ticker	NVDA	0 dtype:int64	Ticker	NVDA	0 dtype:int64
0	21	0.05	Ticker	NVDA	1 dtype:int64	Ticker	NVDA	1 dtype:int64
21	42	0.01	Ticker	NVDA	1 dtype:int64	Ticker	NVDA	1 dtype:int64
21	42	0.05	Ticker	NVDA	2 dtype:int64	Ticker	NVDA	2 dtype:int64
42	63	0.01	Ticker	NVDA	2 dtype:int64	Ticker	NVDA	1 dtype:int64
42	63	0.05	Ticker	NVDA	3 dtype:int64	Ticker	NVDA	2 dtype:int64
63	84	0.01	Ticker	NVDA	0 dtype:int64	Ticker	NVDA	0 dtype:int64
63	84	0.05	Ticker	NVDA	1 dtype:int64	Ticker	NVDA	0 dtype:int64
84	105	0.01	Ticker	NVDA	0 dtype:int64	Ticker	NVDA	0 dtype:int64
84	105	0.05	Ticker	NVDA	1 dtype:int64	Ticker	NVDA	1 dtype:int64
105	126	0.01	Ticker	NVDA	0 dtype:int64	Ticker	NVDA	0 dtype:int64
105	126	0.05	Ticker	NVDA	2 dtype:int64	Ticker	NVDA	2 dtype:int64
126	147	0.01	Ticker	NVDA	0 dtype:int64	Ticker	NVDA	0 dtype:int64
126	147	0.05	Ticker	NVDA	1 dtype:int64	Ticker	NVDA	1 dtype:int64
147	168	0.01	Ticker	NVDA	0 dtype:int64	Ticker	NVDA	0 dtype:int64
147	168	0.05	Ticker	NVDA	0 dtype:int64	Ticker	NVDA	0 dtype:int64
168	189	0.01	Ticker	NVDA	0 dtype:int64	Ticker	NVDA	0 dtype:int64
168	189	0.05	Ticker	NVDA	0 dtype:int64	Ticker	NVDA	0 dtype:int64
189	210	0.01	Ticker	NVDA	0 dtype:int64	Ticker	NVDA	0 dtype:int64
189	210	0.05	Ticker	NVDA	0 dtype:int64	Ticker	NVDA	0 dtype:int64
210	231	0.01	Ticker	NVDA	0 dtype:int64	Ticker	NVDA	1 dtype:int64
210	231	0.05	Ticker	NVDA	1 dtype:int64	Ticker	NVDA	1 dtype:int64
231	252	0.01	Ticker	NVDA	0 dtype:int64	Ticker	NVDA	0 dtype:int64
231	252	0.05	Ticker	NVDA	0 dtype:int64	Ticker	NVDA	0 dtype:int64
252	273	0.01	Ticker	NVDA	0 dtype:int64	Ticker	NVDA	0 dtype:int64
252	273	0.05	Ticker	NVDA	2 dtype:int64	Ticker	NVDA	2 dtype:int64
273	294	0.01	Ticker	NVDA	0 dtype:int64	Ticker	NVDA	0 dtype:int64
273	294	0.05	Ticker	NVDA	0 dtype:int64	Ticker	NVDA	0 dtype:int64
294	315	0.01	Ticker	NVDA	0 dtype:int64	Ticker	NVDA	0 dtype:int64
294	315	0.05	Ticker	NVDA	0 dtype:int64	Ticker	NVDA	0 dtype:int64
315	336	0.01	Ticker	NVDA	0 dtype:int64	Ticker	NVDA	0 dtype:int64
315	336	0.05	Ticker	NVDA	3 dtype:int64	Ticker	NVDA	4 dtype:int64
336	357	0.01	Ticker	NVDA	0 dtype:int64	Ticker	NVDA	0 dtype:int64
336	357	0.05	Ticker	NVDA	0 dtype:int64	Ticker	NVDA	0 dtype:int64
357	378	0.01	Ticker	NVDA	0 dtype:int64	Ticker	NVDA	0 dtype:int64
357	378	0.05	Ticker	NVDA	0 dtype:int64	Ticker	NVDA	0 dtype:int64
378	399	0.01	Ticker	NVDA	0 dtype:int64	Ticker	NVDA	0 dtype:int64
378	399	0.05	Ticker	NVDA	0 dtype:int64	Ticker	NVDA	0 dtype:int64
399	420	0.01	Ticker	NVDA	0 dtype:int64	Ticker	NVDA	0 dtype:int64
399	420	0.05	Ticker	NVDA	2 dtype:int64	Ticker	NVDA	1 dtype:int64
420	441	0.01	Ticker	NVDA	0 dtype:int64	Ticker	NVDA	0 dtype:int64
420	441	0.05	Ticker	NVDA	0 dtype:int64	Ticker	NVDA	0 dtype:int64
441	462	0.01	Ticker	NVDA	1 dtype:int64	Ticker	NVDA	1 dtype:int64
441	462	0.05	Ticker	NVDA	1 dtype:int64	Ticker	NVDA	1 dtype:int64
462	483	0.01	Ticker	NVDA	0 dtype:int64	Ticker	NVDA	0 dtype:int64
462	483	0.05	Ticker	NVDA	0 dtype:int64	Ticker	NVDA	0 dtype:int64
483	504	0.01	Ticker	NVDA	2 dtype:int64	Ticker	NVDA	0 dtype:int64
483	504	0.05	Ticker	NVDA	2 dtype:int64	Ticker	NVDA	2 dtype:int64
504	525	0.01	Ticker	NVDA	0 dtype:int64	Ticker	NVDA	0 dtype:int64
504	525	0.05	Ticker	NVDA	5 dtype:int64	Ticker	NVDA	6 dtype:int64
525	546	0.01	Ticker	NVDA	0 dtype:int64	Ticker	NVDA	0 dtype:int64
525	546	0.05	Ticker	NVDA	1 dtype:int64	Ticker	NVDA	1 dtype:int64
546	567	0.01	Ticker	NVDA	0 dtype:int64	Ticker	NVDA	0 dtype:int64
546	567	0.05	Ticker	NVDA	0 dtype:int64	Ticker	NVDA	0 dtype:int64
567	588	0.01	Ticker	NVDA	0 dtype:int64	Ticker	NVDA	0 dtype:int64
567	588	0.05	Ticker	NVDA	2 dtype:int64	Ticker	NVDA	2 dtype:int64

Figura 28: Resumo das violações observadas para os modelos GARCH ($N(0,1)$ e $t(0,1,m)$).

6.2 Testes de Kupiec e Christoffersen

Os resultados dos testes de Kupiec e Christoffersen são apresentados nas Tabelas 3 e 4, respectivamente, relativos ao resultado da execução do código que é exibido na imagem 29.

```

Teste de Kupiec
Alpha: 0.01, Modelo: Normal, LR_UC: 258.9775, p-valor: 0.0000
Alpha: 0.01, Modelo: T, LR_UC: 228.5664, p-valor: 0.0000
Alpha: 0.05, Modelo: Normal, LR_UC: 144.7477, p-valor: 0.0000
Alpha: 0.05, Modelo: T, LR_UC: 124.2407, p-valor: 0.0000

Teste de Christoffersen
Alpha: 0.01, Modelo: Normal, LR_CC: -0.0025, p-valor: 1.0000
Alpha: 0.01, Modelo: T, LR_CC: -0.4117, p-valor: 1.0000
Alpha: 0.05, Modelo: Normal, LR_CC: -0.0025, p-valor: 1.0000
Alpha: 0.05, Modelo: T, LR_CC: -0.4117, p-valor: 1.0000

```

Figura 29: Saída do programa: Resultados dos testes de Kupiec e Christoffersen.

6.2.1 Teste de Kupiec

O teste de Kupiec avalia se a proporção de violações observadas coincide com o nível de significância (α). A Tabela 3 apresenta os valores da estatística LR_{UC} e os respectivos p -valores.

Tabela 3: Resultados do Teste de Kupiec.

Alpha	Modelo	LR_{UC}	p -valor
0.01	Normal	258.9775	0.0000
0.01	t	228.5664	0.0000
0.05	Normal	144.7477	0.0000
0.05	t	124.2407	0.0000

Os p -valores extremamente baixos indicam que rejeitamos a hipótese nula (H_0) para todos os casos, sugerindo que a proporção de violações observadas não é consistente com o nível de significância esperado.

6.2.2 Teste de Christoffersen

O teste de Christoffersen avalia tanto a cobertura quanto a independência das violações, considerando uma cadeia de Markov. A Tabela 4 apresenta os valores da estatística LR_{CC} e os respectivos p -valores.

Tabela 4: Resultados do Teste de Christoffersen.

Alpha	Modelo	LR_{CC}	p -valor
0.01	Normal	-0.0025	1.0000
0.01	t	-0.4117	1.0000
0.05	Normal	-0.0025	1.0000
0.05	t	-0.4117	1.0000

Os p -valores iguais a 1 indicam que não rejeitamos a hipótese nula (H_0), sugerindo que tanto a cobertura quanto a independência das violações são adequadas.

6.3 Conclusão

Com base nos resultados, concluímos que:

- O teste de Kupiec revela inconsistências na proporção de violações observadas para ambos os modelos e níveis de significância. Isso sugere que os modelos GARCH subestimaram ou superestimaram o VaR para as condições de mercado avaliadas.
- O teste de Christoffersen, por outro lado, não rejeitou a hipótese nula, indicando que as violações do VaR são independentes e seguem uma distribuição consistente com os pressupostos dos modelos.

Os resultados sugerem que, embora os modelos apresentem uma estrutura de dependência adequada (segundo Christoffersen), suas capacidades de cobertura (segundo Kupiec) são limitadas, exigindo possíveis ajustes nos parâmetros ou na modelagem da volatilidade.

7 TAREFA 4: Cálculo do \$VaR em $t = T + 1$

Nesta etapa, foi realizado o cálculo do \$VaR em $t = T + 1$, onde T é o período da última observação da série de retornos. Este cálculo foi baseado nos modelos GARCH estimados anteriormente, utilizando dois níveis de significância: $\alpha = 1\%$ e $\alpha = 5\%$. A Figura 30 apresenta o *output* gerado pelo programa, contendo os resultados detalhados dos valores de \$VaR tanto em termos percentuais quanto monetários.

	Alpha	VaR_Normal	VaR_Normal_Monetary	VaR_T	VaR_T_Monetary
0	0.01	-5.696125	-28480.626873	-5.756821	-28784.106278
1	0.05	-3.921434	-19607.170689	-3.985823	-19929.116647

Figura 30: Saída do programa - Dataframe com os resultados do \$VaR em $t = T + 1$.

7.1 Metodologia

O cálculo foi realizado considerando os seguintes passos:

1. **Modelos GARCH:** Utilizamos os melhores modelos identificados na Tarefa 2:
 - Modelo GARCH(2, 2) com distribuição Normal.
 - Modelo GARCH(1, 1) com distribuição t.
2. **Probabilidades:** Dois níveis de confiança foram considerados:
 - $\alpha = 1\%$, correspondente a um nível de confiança de 99%.
 - $\alpha = 5\%$, correspondente a um nível de confiança de 95%.
3. **Estimativa do \$VaR:**
 - Para cada modelo, foi calculado o \$VaR percentual, utilizando a média condicional ($E_t[R]$) e o desvio padrão condicional (σ_t) fornecidos pelo modelo.
 - Em seguida, o \$VaR foi convertido para valores monetários, assumindo uma posição de 5.000 ações da empresa analisada.

7.2 Resultados

Os resultados obtidos estão apresentados na Figura 30. Seguem os principais pontos observados:

- ***VaR* Percentual:**

- Para $\alpha = 1\%$: O *VaR_{Normal}* foi de -5.70% enquanto o *VaR_T* foi de -5.76% . O modelo t apresentou um *VaR* ligeiramente maior, refletindo suas caudas mais pesadas.
- Para $\alpha = 5\%$: O *VaR_{Normal}* foi de -3.92% e o *VaR_T* foi de -3.99% . Novamente, o modelo t capturou um risco marginalmente maior, como esperado.

- ***\$VaR* Monetário:**

- Para $\alpha = 1\%$: As perdas projetadas foram de \$28.480 (Normal) e \$28.784 (t).
- Para $\alpha = 5\%$: As perdas projetadas foram de \$19.607 (Normal) e \$19.929 (t).

7.3 Escolha do *\$VaR* com Base no Back Test

Com base nos resultados do *back test*, apresentados na Seção 6, a escolha do *\$VaR* para cada valor de α leva em consideração a adequação dos modelos avaliados em termos de cobertura e independência das violações. A seguir, justificamos a escolha:

- **Para $\alpha = 1\%$:**

- O modelo GARCH com distribuição t apresentou melhor desempenho no teste de Christoffersen, pois capturou adequadamente as violações em termos de independência e sequência de eventos, conforme evidenciado pelo p -valor elevado do teste ($p = 1.000$).
- Apesar do modelo com distribuição Normal também apresentar um p -valor elevado no teste de Christoffersen, o modelo t capturou caudas mais pesadas, o que é essencial para eventos extremos, como o caso de $\alpha = 1\%$.
- Escolha final: Utilizamos o *\$VaR* calculado com o modelo GARCH(1, 1) e distribuição t.

- **Para $\alpha = 5\%$:**

- Similarmente ao caso de $\alpha = 1\%$, o modelo com distribuição t apresentou um desempenho mais robusto, principalmente ao capturar adequadamente o risco de eventos extremos.
- O modelo Normal, embora adequado em alguns aspectos, subestima ligeiramente o risco em níveis mais elevados de α .
- Escolha final: Utilizamos o *\$VaR* calculado com o modelo GARCH(1, 1) e distribuição t.

7.3.1 Justificativa e Interpretação dos Resultados

A escolha do modelo GARCH com distribuição t para ambos os níveis de α é fundamentada na superioridade deste modelo em capturar eventos extremos e violações independentes. Os resultados do *back test* corroboram esta decisão, visto que o modelo com distribuição t apresentou maior capacidade de adequação aos pressupostos do teste de Christoffersen.

Do ponto de vista prático, para uma aplicação de 5.000 ações, os valores monetários associados ao \$VaR calculado fornecem uma estimativa confiável das perdas potenciais, respeitando os níveis de confiança estabelecidos. A escolha de um modelo conservador é crucial para mitigar riscos em cenários de alta volatilidade e incerteza.

7.4 Análise e Discussão

- **Coerência dos Resultados:** Os resultados dos modelos são consistentes com o comportamento esperado das distribuições Normal e t . O modelo com distribuição t captura melhor eventos extremos, resultando em valores de \$VaR ligeiramente mais altos.
- **Implicações Práticas:** Em situações onde é necessário maior conservadorismo na gestão de risco, o modelo t deve ser preferido. No entanto, ambos os modelos demonstraram capacidade de captura adequada de risco em diferentes níveis de confiança.
- **Conclusão:** Com base nos valores calculados, o modelo GARCH com distribuição t é mais adequado para situações onde há maior preocupação com eventos de cauda, como crises de mercado. Para uma abordagem menos conservadora, o modelo Normal pode ser utilizado.

8 Conclusões Gerais

O trabalho teve como objetivo principal a análise de modelos GARCH, considerando distribuições Normal e t , para estimar e avaliar o \$VaR em diferentes níveis de confiança ($\alpha = 1\%$ e $\alpha = 5\%$). Por meio de diversas etapas, incluindo a seleção de modelos, realização de *back tests*, aplicação de testes de diagnóstico e o cálculo do \$VaR em $t = T + 1$, foi possível construir um panorama consistente da performance dos modelos estimados e sua adequação para a gestão de risco financeiro.

Realizamos a estimativa de diversos modelos GARCH(p, q) para cada distribuição ($N(0, 1)$ e $t(0, 1, m)$), utilizando os critérios AIC e BIC para identificar os melhores ajustes. Os resultados indicaram que o modelo GARCH(2, 2) com distribuição Normal e o modelo GARCH(1, 1) com distribuição t apresentaram os melhores desempenhos segundo os critérios de informação. Essa escolha foi validada pelos parâmetros estimados, que indicaram a adequação dos modelos aos dados históricos.

O *back test* foi uma etapa fundamental para avaliar a capacidade preditiva dos modelos em termos de cobertura e independência das violações do \$VaR. Os testes de Kupiec e Christoffersen demonstraram que o modelo com distribuição t foi mais robusto em capturar adequadamente os eventos extremos e as violações independentes, especialmente nos casos mais conservadores ($\alpha = 1\%$). A análise evidenciou que a distribuição t , por

incorporar caudas mais pesadas, é mais apropriada para capturar a dinâmica de riscos extremos em séries financeiras.

A etapa final consistiu no cálculo do \$VaR para $t = T + 1$, considerando uma aplicação de 5.000 ações da empresa analisada. Os valores monetários estimados refletem as perdas potenciais esperadas para os dois níveis de confiança, utilizando tanto o modelo com distribuição Normal quanto o modelo com distribuição t . A escolha do modelo t foi reafirmada como a mais adequada, devido à sua maior precisão em capturar eventos extremos e sua consistência nos *back tests*.

8.1 Contribuições e Limitações

Este trabalho reforça a importância de uma análise criteriosa na seleção de modelos para a gestão de risco. O uso de testes estatísticos, como Kupiec e Christoffersen, permite validar a adequação dos modelos, garantindo que as estimativas de \$VaR sejam confiáveis e consistentes. Entretanto, reconhecemos que o modelo GARCH, apesar de robusto, possui limitações inerentes, como a dificuldade em capturar mudanças abruptas no regime de volatilidade e a dependência de pressupostos como a distribuição dos erros.

8.2 Perspectivas Futuras

Para trabalhos futuros, poderíamos sugerir a exploração de modelos mais avançados, como GARCH multivariados ou modelos com componentes estocásticos, para capturar possíveis correlações entre múltiplos ativos financeiros. Além disso, a inclusão de variáveis macroeconômicas como fatores explicativos pode enriquecer a análise e aumentar a precisão das estimativas de risco.

8.3 Conclusão Final

Em suma, este trabalho demonstrou a relevância dos modelos GARCH para a gestão de risco em mercados financeiros, evidenciando a superioridade do modelo com distribuição t em contextos de alta incerteza e volatilidade. A integração entre teoria e prática, por meio do uso de testes estatísticos e simulações reais, fortalece a confiabilidade das conclusões e contribui para a aplicação de ferramentas quantitativas na tomada de decisão financeira.

9 Apêndice

9.1 Código Python utilizado em 4.1

```
1  """ TAREFA 1 """
2  ### Parte I
3
4  # Cálculo dos retornos logarítmicos e aritméticos
5  # Retornos logarítmicos
6  log_returns = np.log(data['Adj Close'] / data['Adj Close'].
7      shift(1)) * 100
8
9  arith_returns = data['Adj Close'].pct_change() * 100 #
10     Retornos aritméticos
11
12
13 # Removendo valores NaN
14 log_returns.dropna(inplace=True)
15 arith_returns.dropna(inplace=True)
16
17 # Criando uma série para a diferença entre os retornos
18 diff_returns = arith_returns - log_returns
19
20
21 # Plotando os retornos logarítmicos
22 plt.figure(figsize=(14, 7))
23 plt.plot(log_returns, label='Retornos Logarítmicos', color='
24     blue')
25 plt.title('Retornos Logarítmicos')
26 plt.xlabel('Data')
27 plt.ylabel('Retornos (%)')
28 plt.legend()
29 plt.grid()
30 plt.show()
31
32 # Plotando os retornos aritméticos
33 plt.figure(figsize=(14, 7))
34 plt.plot(arith_returns, label='Retornos Aritméticos', color='
35     orange')
36 plt.title('Retornos Aritméticos')
37 plt.xlabel('Data')
38 plt.ylabel('Retornos (%)')
39 plt.legend()
40 plt.grid()
41 plt.show()
42
43 # Plotando os dois retornos para comparação
44 plt.figure(figsize=(14, 7))
45 plt.plot(log_returns, label='Retornos Logarítmicos', color='
46     blue', alpha=0.7)
47 plt.plot(arith_returns, label='Retornos Aritméticos',
48     color='orange', alpha=0.7)
```



```

42 plt.title('Compara o: Retornos Logar tmicos vs. Retornos
    Aritm ticos')
43 plt.xlabel('Data')
44 plt.ylabel('Retornos (%)')
45 plt.legend()
46 plt.grid()
47 plt.show()
48
49 # Plotando a diferen a entre os retornos
50 plt.figure(figsize=(14, 7))
51 plt.plot(diff_returns, label='Diferen a entre Retornos',
    color='red')
52 plt.title('Diferen a entre Retornos Aritm ticos e
    Logar tmicos')
53 plt.xlabel('Data')
54 plt.ylabel('Diferen a (%)')
55 plt.legend()
56 plt.grid()
57 plt.show()
58
59 # Dias com maior diferen a entre os retornos
60 # Extraindo a coluna com os valores de diff_returns
61 diff_abs = diff_returns['NVDA'].abs()
62
63 # Selecionando os 5 maiores desvios
64 top_diff_days = diff_abs.nlargest(10)
65 print(top_diff_days)
66
67
68
69
70 # metodo alternativo
71 data_ = pd.read_csv('nvda_historical_data_.csv', sep=';')
72 data_['Date'] = pd.to_datetime(data_['Date'], format='%d/%m/%
    Y')
73 data_.set_index('Date', inplace=True)
74
75 # Calculando os retornos
76 data_['log_return'] = np.log(data_['Close']/data_['Close'].
    shift(1)) * 100
77 data_['arit_return'] = ((data_['Close']/data_['Close'].shift
    (1)) - 1) * 100
78 data_['difference'] = data_['arit_return'] - data_['
    log_return']
79
80 # Removendo valores NA
81 data_ = data_.dropna()
82
83 # Encontrando os dias com maiores diferencas

```

```

84 threshold = data_['difference'].std() * 2 # 2 desvios
    padr o
85 significant_diff = data_[abs(data_['difference']) > threshold
    ]
86
87
88 # Mostrando os dias com maiores diferencas
89 print("\nDias com diferencas mais significativas (> 2 desvios
    padrao):")
90 print("Data | Ret. Aritmetico | Ret. Logar tmico |
    Diferen a")
91 print("-" * 60)
92 for idx in significant_diff.index:
93     print(f"{idx.date()} | {significant_diff.loc[idx, '
        arit_return']:8.2f}% | {significant_diff.loc[idx, '
        log_return']:8.2f}% | {significant_diff.loc[idx, '
        difference']:8.2f}%")

```

9.2 Código Python utilizado em 4.2

```

1     """TAREFA1"""
2     ### Parte II
3
4     # Dividindo em 70% para treinamento e 30% para back-test
5     train_size = int(0.7 * len(arith_returns))
6     train_data = arith_returns[:train_size]
7     test_data = arith_returns[train_size:]
8
9     print(f'Tamanho do per odo de treinamento: {len(train_data)}
    ')
10    print(f'Tamanho do per odo de back-test: {len(test_data)}')
11
12
13    #Passo 2: grafico da serie de retornos aritmeticos no periodo
    de treinamento
14    plt.figure(figsize=(14, 7))
15    plt.plot(train_data, label='Retornos Aritm ticos (
        Treinamento)', color='blue')
16    plt.title('Retornos Aritmeticos no Per odo de Treinamento')
17    plt.xlabel('Data')
18    plt.ylabel('Retorno (%)')
19    plt.legend()
20    plt.grid()
21    plt.show()
22
23    # Calculando a volatilidade em janela movel (desvio padr o
    de 30 dias)
24    rolling_volatility = train_data.rolling(window=30).std()

```

```

25
26 # Plotando a volatilidade
27 plt.figure(figsize=(14, 7))
28 plt.plot(rolling_volatility, label='Volatilidade (30 dias)',
29         color='red')
30 plt.title('Volatilidade Estimada (Janela Móvel de 30 Dias)')
31 plt.xlabel('Data')
32 plt.ylabel('Volatilidade (%)')
33 plt.legend()
34 plt.grid()
35 plt.show()
36
37 # Plotando os quadrados dos retornos
38 plt.figure(figsize=(14, 7))
39 plt.plot(train_data**2, label='Quadrados dos Retornos', color
40         = 'purple')
41 plt.title('Quadrados dos Retornos (Proxies de Volatilidade)')
42 plt.xlabel('Data')
43 plt.ylabel('Retorno ')
44 plt.legend()
45 plt.grid()
46 plt.show()
47
48 # como train_data é um dataframe, vamos pegar apenas a
49 # coluna necessária para
50 # realizar as estatísticas
51 train_data = train_data['NVDA']
52
53 # Passo 3: estatísticas descritivas
54 # Calculando estatísticas descritivas
55 quantiles = [0.005, 0.01, 0.05, 0.5, 0.95, 0.99, 0.995]
56 stats = {
57     'Mínimo': train_data.min(),
58     'Máximo': train_data.max(),
59     'Média': train_data.mean(),
60     'Desvio Padrão': train_data.std(),
61     'Assimetria': train_data.skew(),
62     'Curtose': train_data.kurtosis(),
63     **{f'Quantil {int(q * 100)}%': train_data.quantile(q) for
64        q in quantiles}
65 }
66
67 # Criando DataFrame para exibir as estatísticas
68 stats_df = pd.DataFrame(stats, index=[0])
69
70 # Exibindo diretamente no console
71 print(stats_df)
72
73 # Salvando a tabela em CSV

```

```

71 stats_df.to_csv("estatisticas_descritivas.csv", index=False)
72 print("Estat sticas salvas em 'estatisticas_descritivas.csv'
    ")
73
74
75 # PASSO 4: FAC e Teste LB para Depend ncia Linear
76 # FAC
77 plot_acf(train_data, lags=20, title='FAC dos Retornos
    Aritm ticos (Treinamento)')
78 plt.show()
79
80 # Teste de Ljung-Box
81 lb_test = acorr_ljungbox(train_data, lags=[10], return_df=
    True)
82 print(lb_test)
83
84
85 # PASSO 5: FAC do Quadrado e Teste LB para Depend ncia N o
    Linear
86 # # FAC dos quadrados dos retornos
87 plot_acf(train_data**2, lags=20, title='FAC dos Quadrados dos
    Retornos (Treinamento)')
88 plt.show()
89
90 # Teste de Ljung-Box para quadrados
91 lb_test_sq = acorr_ljungbox(train_data**2, lags=[10],
    return_df=True)
92 print(lb_test_sq)
93
94
95 # PASSO 6: TESTE ARCH
96 # Teste ARCH
97 arch_test = het_arch(train_data)
98 print(f"Teste ARCH:\nLM Stat: {arch_test[0]}, p-valor: {
    arch_test[1]}")
99
100
101 # PASSO 7: HISTOGRAMA, QQ PLOT e Teste de Normalidade
102
103 # Histograma
104 plt.figure(figsize=(10, 6))
105 plt.hist(train_data, bins=30, color='blue',
    alpha=0.7, density=True, label='Retornos')
106 xmin, xmax = plt.xlim()
107 x = np.linspace(xmin, xmax, 100)
108 plt.plot(x, stats.norm.pdf(x, train_data.mean(),
    train_data.std()), 'r', label='Normal')
109 plt.title('Histograma dos Retornos Aritm ticos')
110 plt.grid(True)
111 plt.legend()
112
113

```

```

114 plt.show()
115
116 # Q-Q Plot
117 stats.probplot(train_data, dist="norm", plot=plt)
118 plt.title('Q-Q Plot dos Retornos Aritméticos')
119 plt.grid(True)
120 plt.show()
121
122 # Teste de Normalidade (Shapiro-Wilk)
123 shapiro_test = stats.shapiro(train_data)
124 print(
125     f"Teste de Shapiro-Wilk: Estatística={shapiro_test[0]}, p
        -valor={shapiro_test[1]}")

```

9.3 Código Python utilizado em 5.5

```

1  """TAREFA 2"""
2
3  # PARTE I
4
5  # Passo 1: Carregar os dados e selecionar o periodo de
        treinamento
6  train_data = train_data #ja foi definido anteriormente
7
8  # Passo 2: Definir os parametros do modelo
9  p_values = [1, 2, 3] # Valores de p (ordem da autoregressao)
10 q_values = [1, 2, 3] # Valores de q (ordem da media movel)
11 distributions = ['normal', 't'] # Distribuicoes: N(0,1) e t
        (0,1,m)
12
13 # DataFrame para armazenar os resultados
14 results = []
15
16 # Passo 3: Ajustar os modelos GARCH(p, q) para cada
        distribuicao
17 for dist in distributions:
18     for p in p_values:
19         for q in q_values:
20             try:
21                 # Estimar o modelo
22                 model = arch_model(train_data, vol='Garch', p
                    =p, q=q, dist=dist)
23                 fitted_model = model.fit(dis="off")
24
25                 # Coletar os resultados
26                 results.append({
27                     'Distribui o': dist,
28                     'p': p,

```

```

29         'q': q,
30         'AIC': fitted_model.aic,
31         'BIC': fitted_model.bic,
32         'Modelo': fitted_model.params
33     })
34     except Exception as e:
35         print(f"Erro ao ajustar modelo GARCH({p}, {q}
36               }) com distribuicao {dist}: {e}")
37
38 # Passo 4: Organizar os resultados em um DataFrame
39 results_df = pd.DataFrame(results)
40
41 # Passo 5: Identificar os melhores modelos (menor AIC e BIC
42           para cada distribuicao)
43 best_models = results_df.loc[results_df.groupby('Distribuicao
44                              ')['AIC'].idxmin()]
45 best_models_bic = results_df.loc[results_df.groupby('
46                               Distribuicao')['BIC'].idxmin()]
47
48 # Passo 6: Salvar a tabela de resultados
49 results_df.to_csv('resultados_garch.csv', index=False)
50 best_models.to_csv('melhores_modelos_aic.csv', index=False)
51 best_models_bic.to_csv('melhores_modelos_bic.csv', index=
52                         False)
53
54 # Exibir os melhores modelos
55 print("Melhores modelos (AIC):")
56 print(best_models)
57 print("\nMelhores modelos (BIC):")
58 print(best_models_bic)
59
60 # Passo 7: Parametros para as equacoes do modelo
61 for index, row in best_models.iterrows():
62     print(f"\nModelo GARCH({row['p']}, {row['q']}) com
63           distribuicao {row['Distribuicao']}:")
64     print(f"Parametros:\n{row['Modelo']}")

```

9.4 Código Python utilizado em 5.6

```

1  """TAREFA 2"""
2  # Parte II
3
4  # Configuracoes gerais
5  plt.style.use('seaborn-v0_8-darkgrid')
6  significance_level = 0.05
7
8  # Função para obter res duos padronizados do modelo
9  def get_standardized_residuals(model):

```

```

10     residuals = model.std_resid
11     return residuals
12
13
14 # Reajustando os modelos
15 normal_model = arch_model(train_data, vol='Garch', p=2, q=2,
16     dist='normal')
17 normal_model_result = normal_model.fit(disp='off')
18 t_model = arch_model(train_data, vol='Garch', p=1, q=1, dist=
19     't')
20 t_model_result = t_model.fit(disp='off')
21
22 # Atualizando os dicionários com os resultados ajustados
23     completos
24 best_normal_model = {'Modelo': normal_model_result}
25 best_t_model = {'Modelo': t_model_result}
26
27 # Resíduos padronizados
28 resid_normal = get_standardized_residuals(best_normal_model['
29     Modelo'])
30 resid_t = get_standardized_residuals(best_t_model['Modelo'])
31
32 # 1. FAC dos resíduos padronizados
33 def plot_fac(residuals, title, filename):
34     plt.figure(figsize=(10, 6))
35     plot_acf(residuals, lags=20, alpha=significance_level)
36     plt.title(title)
37     plt.show()
38
39 plot_fac(resid_normal, "FAC dos Resíduos Padronizados -
40     Normal", "fac_residuos_normal")
41 plot_fac(resid_t, "FAC dos Resíduos Padronizados - t", "
42     fac_residuos_t")
43
44 # 2. FAC dos quadrados dos resíduos padronizados
45 def plot_fac_squared(residuals, title, filename):
46     plt.figure(figsize=(10, 6))
47     plot_acf(residuals ** 2, lags=20, alpha=
48     significance_level)
49     plt.title(title)
50     plt.show()
51
52 plot_fac_squared(resid_normal, "FAC dos Quadrados dos
53     Resíduos Padronizados - Normal", "
54     fac_quadrados_residuos_normal")
55 plot_fac_squared(resid_t, "FAC dos Quadrados dos Resíduos
56     Padronizados - t", "fac_quadrados_residuos_t")
57
58 # 3. Teste ARCH

```

```

50 def test_arch(residuals):
51     arch_model_instance = arch_model(residuals)
52     test_results = arch_model_instance.fit(dis="off").
        arch_lm_test()
53     return test_results
54
55 arch_test_normal = test_arch(resid_normal)
56 arch_test_t = test_arch(resid_t)
57
58 print("Teste ARCH - Normal:")
59 print(
60     f"Estat stica LM: {arch_test_normal.stat}, p-valor: {
        arch_test_normal.pval}")
61
62 print("Teste ARCH - t:")
63 print(f"Estat stica LM: {arch_test_t.stat}, p-valor: {
        arch_test_t.pval}")
64
65 # 4. Q-Q Plot e Teste Jarque-Bera
66 def qq_plot(residuals, title, filename):
67     plt.figure(figsize=(8, 6))
68     probplot(residuals, dist="norm", plot=plt)
69     plt.title(title)
70     plt.show()
71
72 qq_plot(resid_normal, "Q-Q Plot dos Res duos Padronizados -
        Normal", "qqplot_normal")
73 qq_plot(resid_t, "Q-Q Plot dos Res duos Padronizados - t", "
        qqplot_t")
74
75 def jb_test(residuals):
76     return jarque_bera(residuals)
77
78 jb_test_normal = jb_test(resid_normal)
79 jb_test_t = jb_test(resid_t)
80
81 # Exibir resultados do Teste Jarque-Bera
82 print("Teste Jarque-Bera - Normal:")
83 print(f"Estat stica JB: {jb_test_normal[0]}, p-valor: {
        jb_test_normal[1]}")
84
85 print("Teste Jarque-Bera - t:")
86 print(f"Estat stica JB: {jb_test_t[0]}, p-valor: {jb_test_t
        [1]}")

```

9.5 Código Python utilizado em 6


```

1  """TAREFA 3 - Back Test com os Melhores Modelos da Tarefa 2
   """
2
3  # Configuracoes gerais
4  alpha_values = [0.01, 0.05]  # Niveis de significancia para o
   VaR
5  rolling_window = 21  # Janela de reestimativa (21 dias)
6  test_size_ratio = 0.3  # Proporcao de dados para teste
7
8  # 1. Divis o dos dados em treinamento e teste
9  total_obs = len(arith_returns)
10 test_size = int(test_size_ratio * total_obs)
11 train_data = arith_returns[:-test_size]
12 test_data = arith_returns[-test_size:]
13
14 print(f"Total de observa es: {total_obs}")
15 print(f"Tamanho do conjunto de treino: {len(train_data)}")
16 print(f"Tamanho do conjunto de teste: {len(test_data)}")
17
18 # 2. Recuperar os melhores modelos identificados na Tarefa 2
19 garch_normal_params = best_models.loc[best_models['
   Distribui o '] == 'normal']
20 garch_t_params = best_models.loc[best_models['Distribui o '
   ] == 't']
21
22 # Garantir que estamos usando os melhores modelos (ordem p, q
   ) para cada distribuicao
23 p_normal = int(garch_normal_params['p'].values[0])
24 q_normal = int(garch_normal_params['q'].values[0])
25 p_t = int(garch_t_params['p'].values[0])
26 q_t = int(garch_t_params['q'].values[0])
27
28 # 3. Funcao para calcular o VaR com base no modelo GARCH
   reestimado
29
30 def calculate_var(model_result, model, returns, alpha):
31     """
32     Calcula os quantis condicionais de VaR para um modelo
33     GARCH reestimado.
34     """
35     forecast = model_result.forecast(horizon=1, start=len(
36     returns) - 1)
37     cond_mean = forecast.mean.iloc[-1, 0]
38     cond_vol = forecast.variance.iloc[-1, 0]**0.5
39
40     if model.distribution.name == "t":
41         q = model.distribution.ppf(alpha)  # Percentil da t-
           Student
42     else:
43         q = norm.ppf(alpha)  # Percentil da Normal

```

```

42     return cond_mean + q * cond_vol
43
44
45 # 4. Back Test com reestimativa dos modelos
46 results = []
47
48 for i in range(0, len(test_data) - rolling_window + 1,
49               rolling_window):
49     train_subset = np.concatenate([train_data, test_data[:i
50                                   ]])
51     test_subset = test_data[i: i + rolling_window]
52
53     # Reestimativa do modelo GARCH para distribuicao normal
54     garch_normal = arch_model(
55         train_subset, vol="Garch", p=p_normal, q=q_normal,
56         dist="normal")
57     garch_normal_result = garch_normal.fit(dispatch="off")
58
59     # Reestimativa do modelo GARCH para distribuicao t
60     garch_t = arch_model(train_subset, vol="Garch", p=p_t, q=
61                           q_t, dist="t")
62     garch_t_result = garch_t.fit(dispatch="off")
63
64     for alpha in alpha_values:
65         # Calcular o VaR para ambos os modelos
66         var_normal = calculate_var(
67             garch_normal_result, garch_normal, train_subset,
68             alpha)
69         var_t = calculate_var(garch_t_result, garch_t,
70                               train_subset, alpha)
71
72         # Verificar viola es
73         violations_normal = (test_subset < var_normal).sum()
74         violations_t = (test_subset < var_t).sum()
75
76         # Salvar resultados
77         results.append({
78             "Janela Inicial": i,
79             "Janela Final": i + rolling_window,
80             "Alpha": alpha,
81             "Violations_Normal": violations_normal,
82             "Violations_T": violations_t,
83         })
84
85 # 5. Gerar DataFrame com os resultados
86 results_df = pd.DataFrame(results)
87
88 # Exibir os resultados
89 print("Resultados do Back Test:")
90 print(results_df)

```

```

86
87 # Pr ximos passos: realizar os testes de Kupiec e
    Christoffersen usando `results_df`.
88 # Funcao para calcular o teste de Kupiec
89
90
91 def kupiec_test(violations, alpha, total_windows):
92     """
93     Realiza o teste de Kupiec para cobertura incondicional.
94     """
95     n_1 = np.sum(violations) # N mero de violacoes
96     n_0 = total_windows - n_1 # N mero de n o violacoes
97     pi_hat = n_1 / total_windows # Proporcao observada
98
99     # Raz o de verossimilhanca
100     lr_uc = -2 * (
101         n_0 * np.log(1 - alpha)
102         + n_1 * np.log(alpha)
103         - (n_0 * np.log(1 - pi_hat) + n_1 * np.log(pi_hat))
104     )
105     # Qui-quadrado com 1 grau de liberdade
106     p_value = 1 - stats.chi2.cdf(lr_uc, df=1)
107     return lr_uc, p_value
108
109 # Fun o para calcular o teste de Christoffersen
110 def christoffersen_test(violations):
111     """
112     Realiza o teste de Christoffersen para cobertura
        condicional.
113     """
114     # Contar transi es observadas
115     n_00 = np.sum((violations[:-1] == 0) & (violations[1:] ==
        0))
116     n_01 = np.sum((violations[:-1] == 0) & (violations[1:] ==
        1))
117     n_10 = np.sum((violations[:-1] == 1) & (violations[1:] ==
        0))
118     n_11 = np.sum((violations[:-1] == 1) & (violations[1:] ==
        1))
119
120     # Transicoes observadas
121     n_0 = n_00 + n_01
122     n_1 = n_10 + n_11
123
124     # Proporcoes estimadas
125     pi_0 = n_01 / n_0 if n_0 > 0 else 0
126     pi_1 = n_11 / n_1 if n_1 > 0 else 0
127     pi_hat = (n_01 + n_11) / (n_0 + n_1)
128
129     # Raz o de verossimilhan a

```

```

130     lr_cc = -2 * (
131         n_00 * np.log(1 - pi_0)
132         + n_01 * np.log(pi_0)
133         + n_10 * np.log(1 - pi_1)
134         + n_11 * np.log(pi_1)
135         - (n_00 + n_10) * np.log(1 - pi_hat)
136         - (n_01 + n_11) * np.log(pi_hat)
137     )
138     # Qui-quadrado com 2 graus de liberdade
139     p_value = 1 - stats.chi2.cdf(lr_cc, df=2)
140     return lr_cc, p_value
141
142
143 # Aplica o dos testes
144 alpha_values = [0.01, 0.05] # N veis de significancia
145 results_kupiec = []
146 results_christoffersen = []
147
148 for alpha in alpha_values:
149     # Certifique-se de que estamos trabalhando com inteiros
150     violations_normal = results_df["Violations_Normal"].
151         astype(int).to_numpy()
152     violations_t = results_df["Violations_T"].astype(int).
153         to_numpy()
154
155     # Teste de Kupiec
156     kupiec_normal = kupiec_test(
157         violations_normal, alpha, len(violations_normal))
158     kupiec_t = kupiec_test(violations_t, alpha, len(
159         violations_t))
160     results_kupiec.append((alpha, "Normal", *kupiec_normal))
161     results_kupiec.append((alpha, "T", *kupiec_t))
162
163     # Teste de Christoffersen
164     christoffersen_normal = christoffersen_test(
165         violations_normal)
166     christoffersen_t = christoffersen_test(violations_t)
167     results_christoffersen.append((alpha, "Normal", *
168         christoffersen_normal))
169     results_christoffersen.append((alpha, "T", *
170         christoffersen_t))
171
172 # Exibindo resultados
173 print("Teste de Kupiec")
174 for result in results_kupiec:
175     print(
176         f"Alpha: {result[0]}, Modelo: {result[1]}, LR_UC: {
177             result[2]:.4f}, p-valor: {result[3]:.4f}"
178     )

```

```

173 print("\nTeste de Christoffersen")
174 for result in results_christoffersen:
175     print(
176         f"Alpha: {result[0]}, Modelo: {result[1]}, LR_CC: {
177             result[2]:.4f}, p-valor: {result[3]:.4f}"
178     )

```

9.6 Código Python utilizado em 7

```

1  """TAREFA 4"""
2
3
4  # Melhor modelo com distribui o Normal
5  best_normal_model = best_models.loc[best_models["
6      Distribui o"] == "normal"]
7  p_normal = int(best_normal_model["p"].values[0])
8  q_normal = int(best_normal_model["q"].values[0])
9
10 # Melhor modelo com distribui o t
11 best_t_model = best_models.loc[best_models["Distribui o"]
12     == "t"]
13 p_t = int(best_t_model["p"].values[0])
14 q_t = int(best_t_model["q"].values[0])
15
16 # ltima observa o dos dados de retorno
17 final_train_data = arith_returns
18
19 # Ajustar os modelos para obter os par metros finais
20 garch_normal = arch_model(final_train_data, vol="Garch",
21     p=p_normal, q=q_normal, dist="
22     normal")
23 garch_normal_result = garch_normal.fit(dis="off")
24
25 garch_t = arch_model(final_train_data, vol="Garch", p=p_t, q=
26     q_t, dist="t")
27 garch_t_result = garch_t.fit(dis="off")
28
29 # Fun o para calcular o VaR em t = T+1
30
31 def calculate_var_t1(model_result, model, alpha):
32     forecast = model_result.forecast(
33         horizon=1, start=len(final_train_data) - 1)
34     cond_mean = forecast.mean.iloc[-1, 0]
35     cond_vol = np.sqrt(forecast.variance.iloc[-1, 0])
36
37     if model.distribution.name == "t":

```

```

35         df = model_result.params["nu"] # Graus de liberdade
           da t-Student
36         q = model.distribution.ppf(alpha, df)
37     else:
38         q = norm.ppf(alpha)
39
40     return cond_mean + q * cond_vol
41
42
43 # Valores de significância (alpha) e a es
44 alpha_values = [0.01, 0.05]
45 investment = 5000 # Supondo 5.000 a es da empresa
46
47 # Calcular o VaR para cada modelo e cada alpha
48 results_var = []
49
50 for alpha in alpha_values:
51     var_normal = calculate_var_t1(garch_normal_result,
52                                   garch_normal, alpha)
53     var_t = calculate_var_t1(garch_t_result, garch_t, alpha)
54
55     # Calcular impacto monetário
56     monetary_var_normal = investment * var_normal
57     monetary_var_t = investment * var_t
58
59     # Armazenar os resultados
60     results_var.append({
61         "Alpha": alpha,
62         "VaR_Normal": var_normal,
63         "VaR_Normal_Monetary": monetary_var_normal,
64         "VaR_T": var_t,
65         "VaR_T_Monetary": monetary_var_t
66     })
67
68 # Transformar os resultados em DataFrame
69 results_var_df = pd.DataFrame(results_var)
70
71 # Exibir os resultados
72 print("Resultados do VaR em t = T+1:")
73 print(results_var_df)
74
75 # Salvar os resultados em um arquivo CSV
76 results_var_df.to_csv("VaR_results_t1.csv", index=False)

```